VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI– 590018, KARNATAKA, INDIA

A PROJECT REPORT

on

"Swaraaha: Stutter Classification and Localization System"

Submitted in partial fulfilment of the requirements for the award of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted By

Name	USN
K SHREEKRISHNA UPADHYAYA	4VP23AI020
M CHETHAN KESHAV BHAT	4VP23AI023
SKANDA PRASAD K	4VP23AI051
SRINIVAS HEGDE M	4VP23AI054

Under the Guidance of

Prof. Ajay Shastry C G

Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt., of Karnataka

Nehru Nagar, Puttur - 574 203, DK, Karnataka, India.

November, 2026

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka

Nehru Nagar, Puttur - 574203, DK, Karnataka, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

Certified that the project work entitled **"Swaraaha: Stutter Classification and Localization System"** is carried out by **Mr. K Shreekrishna Upadhyaya, Mr. M Chethan Keshav Bhat, Mr. Skanda Prasad K, Mr. Srinivas Hegde M** bearing USNs **4VP23AI020**, **4VP23AI023, 4VP23AI051 and 4VP23AI054** respectively bonafide students of **Vivekananda College of Engineering & Technology, Puttur** in partial fulfilment for the award of **Bachelor of Engineering** in **Artificial Intelligence & Machine Learning** of the **Visvesvaraya Technological University**, Belagavi during the year 2026-27. It is certified that all corrections/suggestions indicated during Internal Assessment have been incorporated in the report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of the Guide	Signature of the HOD	Signature of the Principal
Prof. Ajay Shastry C G	**Dr. Radhika Shetty D S**	**Dr. Purushothama Chippar**

EXTERNAL VIVA

Name of the Examiners	Signature with date
1…….....	.....
2…….....	.....

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]
Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka
Nehru Nagar, Puttur - 574203, DK, Karnataka, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that **"Swaraaha: Stutter Classification and Localization System"** submitted by **Mr. K Shreekrishna Upadhyaya, Mr. M Chethan Keshav Bhat, Mr. Skanda Prasad K, Mr. Srinivas Hegde M** bearing USNs **4VP23AI020, 4VP23AI023, 4VP23AI051 and 4VP23AI054** to the Visvesvaraya Technological University, Belagavi, in partial fulfillment for the award of the degree of **Bachelor of Engineering** in **Artificial Intelligence & Machine Learning** is a *bonafide* record of project work carried out by them under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree.

_____

Signature of the Guide
Prof. Ajay Shastry C G
Department of AI & ML
VCET Puttur

DECLARATION

We, **Mr. K Shreekrishna Upadhyaya (4VP23AI020), Mr. M Chethan Keshav Bhat (4VP23AI023), Mr. Skanda Prasad K (4VP23AI051), Mr. Srinivas Hegde M (4VP23AI054)** students of B.E. 7th Semester in Artificial Intelligence & Machine Learning, **Vivekananda College of Engineering & Technology**, Puttur, hereby declare that the project work entitled **"Swaraaha: Stutter Classification and Localization System"** has been carried out by us at VCET, Puttur, under the guidance of **Prof. Ajay Shastry C G,** Assistant Professor, Department of Artificial Intelligence & Machine Learning, Vivekananda College of Engineering & Technology, Puttur, and submitted in partial fulfilment of the requirements for the award of degree in **Bachelor of Engineering in Artificial Intelligence & Machine Learning** by **Visvesvaraya Technological University**, Belagavi during the academic year 2026-2027.

Name of the students	USN	Signature with date
K Shreekrishna Upadhyaya	4VP23AI020	
M Chethan Keshav Bhat	4VP23AI023	
Skanda Prasad K	4VP23AI051	
Srinivas Hegde M	4VP23AI054	

Date:

Place: Puttur

ACKNOWLEDGEMENT

We take this opportunity to express our deep heartfelt gratitude to all those people who have helped us in the successful completion of the project.

First and foremost, we would like to express our sincere gratitude to our guide, **Prof. Ajay Shastry C G.** for providing excellent guidance, encouragement and inspiration throughout the project work. Without his invaluable guidance, this work would never have been a successful one.

We would like to express our sincere gratitude to Head of the Department of Artificial Intelligence & Machine Learning, **Dr. Radhika Shetty D S** for her guidance and inspiration.

We would like to thank our Principal, **Dr. Purushothama Chippar.** for providing all the facilities and a proper environment to work in the college campus.

We would like to thank our management for providing the necessary infrastructure to carry out the project work.

We are thankful to all the teaching and non-teaching staff members of Artificial Intelligence & Machine Learning Department for their help and needed support rendered throughout the project.

ABSTRACT

Stuttering disrupts the natural flow of communication through involuntary repetitions, prolongations, blocks, and pauses. Roughly 1% of people worldwide are affected, and the condition can reach into their social, professional, and personal life. Diagnosis today still relies mostly on manual assessment by speech-language pathologists, which is subjective, time-consuming, and not available to everyone. This project builds an automated system that uses deep learning to detect, classify, and analyze stuttering events in speech recordings. The system identifies the types of dysfluencies present and reports when they occur. Users record or upload audio through a simple interface and receive the results in an accessible format. The result is a tool for early stutter detection and objective speech evaluation, useful to both speech-language pathologists and people in treatment. The system also contributes to the growing use of artificial intelligence in healthcare, supporting diagnosis, therapy, and an improved quality of life for people who stutter.

TABLE OF CONTENTS

Title	Page No.
LIST OF TABLES	I
LIST OF FIGURES	II
LIST OF ABBREVIATIONS	III
CHAPTER 1 INTRODUCTION	**01**
1.1 INTRODUCTION TO THE PROJECT	01
1.2 PROPOSED METHOD	01
1.3 LITERATURE SURVEY	02
1.4 PROBLEM STATEMENT	14
1.5 OBJECTIVES	14
CHAPTER 2 REQUIREMENT SPECIFICATION AND ANALYSIS	**16**
2.1 INTRODUCTION	16
2.2 FUNCTIONAL REQUIREMENTS	16
2.2.1 User Requirements	17
2.2.2 System Requirements	17
2.3 NON-FUNCTIONAL REQUIREMENTS	17
2.3.1 Performance	18
2.3.2 Scalability	18
2.3.3 Availability	18
2.3.4 Usability	19
2.4 Software Requirements	19
2.5 Hardware Requirements	20
CHAPTER 3 SYSTEM DESIGN	**21**
3.1 INTRODUCTION	21
3.2 SYSTEM ARCHITECTURE AND DESIGN	21
3.3 FLOWCHART	24
3.4 USE CASE DIAGRAM AND DESCRIPTIONS	27
3.5 WORKFLOW DIAGRAM	29
3.6 SEQUENCE DIAGRAM	30
CHAPTER 4 SYSTEM IMPLEMENTATION	**33**
4.1 INTRODUCTION	33
4.2 ALGORITHM	33
4.3 IMPLEMENTATION REQUIREMENTS	34
4.3.1 Train the Model	35

4.3.2 Test the Model 35
4.3.3 Model Evaluation 36
4.4 USER INTERFACE 37
CHAPTER 5 SYSTEM TESTING 39
5.1 INTRODUCTION 39
5.2 TYPES OF TESTS 39
5.2.1 Unit Testing 39
5.2.2 Integration Testing 40
5.2.3 Functional Testing 40
5.2.4 System Testing 40
5.3 RESULT AND ANALYSIS 41
5.3.1 Model Evaluation of the Wav2Vec2 Classifiers 41
5.3.2 Wav2Vec2 Classifier Training 45
5.3.3 Model Evaluation of the CNN-based Models 46
5.3.4 CNN-based Model Training 47
5.3.5 Hyper Parameter Tuning 47
5.3.6 Performance Analysis of the Models 48
CHAPTER 6 SCREENSHOTS 50
CHAPTER 7 CONCLUSION AND SCOPE FOR FUTURE ENHANCEMENTS 53
7.1 CONCLUSION 53
7.2 SCOPE FOR FUTURE ENHANCEMENTS 54
REFERENCES 55
PERSONAL PROFILE 59

LIST OF TABLES

Table No.	Title	Page No.
1.1	Summary of Literature Survey	09
5.1	Multitask Wav2Vec2 classifier results	41
5.2	Wav2Vec2 localizer evaluation results	44
5.3	Threshold optimization results	47
5.4	Comparative study of seven architecture variants	48

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	System Architecture	22
3.2	Flowchart	26
3.3	Use Case Diagram	28
3.4	Workflow Diagram	29
3.5	Sequence Diagram	31
5.1	Per-class F1 of the multitask Wav2Vec2 classifier	42
5.2	Per-class AUROC and AUPRC of the multitask Wav2Vec2 classifier	42
5.3	Confusion matrices for the five binary classifiers	43
5.4	Frame-level metrics of the Wav2Vec2 localizer	45
5.5	Per-class tuned F1 on the Boli set	49
6.1	Home page of the web interface	50
6.2	Results page of the web interface	50
6.3	Home page of the desktop application	51
6.4	Analysis result page of the desktop application	52

LIST OF ABBREVIATIONS

Abbreviation	Description
ASR	Automatic Speech Recognition
AUROC	Area Under the Receiver Operating Characteristic Curve
AUPRC	Area Under the Precision-Recall Curve
BERT	Bidirectional Encoder Representations from Transformers
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HMM	Hidden Markov Model
HuBERT	Hidden-Unit BERT
isWER	Intended Speech Word Error Rate
LLM	Large Language Model
LPC	Linear Predictive Coding
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
PCA	Principal Component Analysis
PWS	People Who Stutter
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SEP-28K	Stuttering Event Detection Dataset (28,000 samples)
SLP	Speech-Language Pathologist
SOTA	State of the Art
SVM	Support Vector Machine
TTS	Text-to-Speech
UAR	Unweighted Average Recall
UI	User Interface
VITS	Variational Inference with Adversarial Learning for end-to-end Text-to-Speech
WavLM	Wave Language Model
WER	Word Error Rate
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO THE PROJECT

Speech fluency lets people express thoughts and emotions clearly. Stuttering, a common speech disorder, interrupts that flow with involuntary repetitions, prolongations, or blocks in speech production. It affects people of all ages and can hurt their confidence, their ability to communicate, and their social interactions. Identifying stuttering patterns early helps with therapy and management.

Traditionally, detection and assessment rely on manual observation by speech-language pathologists. The results can be subjective, time-consuming, and inconsistent from one expert to another, and the work takes a physical toll on the pathologist. That dependence on human evaluation makes large-scale assessment and continuous monitoring difficult. With recent advances in deep learning and speech signal processing, automated stutter detection has become a reliable, scalable alternative. This project develops a system that automatically detects, classifies, and analyzes stuttering types (prolongation, block, sound repetition, word repetition, and interjection) using deep learning.

Input audio is preprocessed and converted into embeddings, then analyzed with Wav2Vec 2.0, which captures both local acoustic patterns and long-range temporal dependencies that separate normal from dysfluent speech. Alongside classification, the same audio passes through a localization module that returns the timestamps of the stuttering events, and the detected stutter timestamps are hard-aligned with the transcript timestamps so that only the affected speech segments are identified. The finished model is integrated into a simple interface where users record or upload speech samples and receive an analysis with timestamped detection results and classification output.

The project aims to give speech-language pathologists and people who stutter an accurate, objective, and accessible diagnostic tool. The system also contributes to the growing use of artificial intelligence in healthcare, supporting diagnosis, therapy, and an improved quality of life for people who stutter.

1.2 PROPOSED METHOD

The proposed method aims to develop an efficient deep learning-based system for the early detection and classification of speech dysfluencies from audio recordings. The process begins with comprehensive audio preprocessing that includes noise reduction,

normalization, and enhancement of acoustic clarity across all samples. Segmentation isolates the meaningful regions of speech, eliminating irrelevant silence and background, and data augmentation such as noise injection, time and pitch shifting, and scaling simulates variations in speaking rate and recording conditions. The system accurately categorizes the input into five dysfluency types: prolongation, block, sound repetition, word repetition, and interjection, and localizes the segments where these occur, supporting early diagnosis and effective treatment planning. This automated approach reduces manual workload, minimizes observer bias, and enhances diagnostic reliability, making it a valuable tool in computer-aided speech assessment.

1.3 LITERATURE SURVEY

P. Arbajian et al. [1] A speech segment selection strategy for stutter block detection was studied in this paper. The authors extract acoustic features from annotated speech samples and test machine learning classifiers under different segmentation setups, comparing windowing approaches. The results show that segment choice strongly affects accuracy: some segment lengths model dysfluency patterns better, while bad segmentation drops important temporal information and degrades performance. Segmentation choices therefore matter for detection accuracy and for remediation systems, which the proposed system takes into account in its preprocessing.

V. Mitra et al. [2] This study developed a tuning-based approach that makes a voice assistant handle dysfluent speech better by optimizing an existing hybrid ASR system. The goal is to reduce recognition errors caused by stuttering, such as repetitions and unintended insertions. The method changes the key decoding parameters of the ASR system, giving more weight to the language model by raising the penalty for inserted words and lowering the acoustic model weight. That helps the system filter repeated or disfluent segments and focus on meaningful speech. Tested on speech from 18 participants with varying stuttering severity, it reduced the intended speech Word Error Rate (isWER) by 24% relative. The results show that parameter tuning alone can reduce dysfluency errors without restructuring the ASR architecture, which is why the proposed system keeps model tuning and post-processing in mind.

P. Mohapatra et al. [3] disfluencyNet, a deep learning model that detects speech disfluencies using contextual representations, was introduced in this paper. The model feeds contextual embeddings into a classification network, modeling speech dependencies to catch disfluencies such as repetitions and pauses. Data distillation limits the need for large training sets. Trained and evaluated on SEP-28k and FluencyBank, the model matched

baseline accuracy using only a quarter of the data, which points to efficient training and strong generalization. That supports the proposed system's use of contextual embeddings and efficient training where annotated data is scarce.

J. Liu et al. [4] A method for detecting disfluencies in multilingual speech of varying lengths was proposed in this paper using the context-based embeddings of wav2vec 2.0. A classification network takes the embeddings, and data distillation keeps only high-quality audio fragments where three human annotators agree on the disfluency labels. Tests on multilingual datasets with different speech lengths showed better performance than other approaches. The work supports the proposed system's use of pretrained models like wav2vec 2.0 and data filtering to improve robustness and cross-speaker generalization.

A. Romana et al. [5] The study introduced a multimodal approach that detects speech disfluencies directly from untranscribed audio. A Bi-LSTM fusion model runs at the frame level, combining WavLM acoustic features with BERT language representations drawn from transcripts produced by a fine-tuned Whisper model. This captures both low-level speech patterns and high-level context, and it copes with ASR transcription errors and misalignment. The multimodal setup proved more robust and accurate than single-modality approaches in noisy real-world conditions. That supports combining acoustic and language features in the proposed system.

D. Wagner et al. [6] This paper presented a hybrid approach that uses Large Language Models (LLMs) to combine acoustic and linguistic representations for dysfluency detection. Acoustic features come from wav2vec 2.0 and transcriptions from Whisper ASR; the two are fused into a joint input that an LLM classifies into stutter types including repetitions, prolongations, and blocks. The acoustic-lexical combination outperformed models that use only audio or only text. The result backs the proposed system's plan to combine audio and language signals for better accuracy and generalization.

S. A. Sheikh et al. [7] The study developed a deep learning framework called multi-contextual (MC) StutterNet for stuttering detection that tackles class imbalance and limited data. The multi-branch architecture processes different contextual representations of speech. Class-balanced loss assigns more weight to underrepresented stutter categories, and data augmentation diversifies the training set. Results showed more robust and accurate detection across speech conditions, especially for less frequent dysfluency types. The proposed system uses the same ideas: data augmentation, balanced loss, and multi-context learning.

X. Zhou et al. [8] A YOLO-inspired model that treats spectrograms as images was introduced in this paper, detecting dysfluencies with simultaneous localization and classification in the time-frequency domain. A region-based detection model learns spatial features (frequency patterns) and temporal dynamics (changes over time) from these images. Speech-text alignment links audio segments to their corresponding words for better context. The model predicts both the dysfluency type and the exact time region where it occurs, with strong results across multiple dysfluency types. This is why the proposed system builds its real-time, region-wise detection on spectrogram-based CNN architectures.

X. Zhou et al. [9] stutter-Solver, an end-to-end YOLO-inspired model for multilingual dysfluency detection, was developed in this paper. It treats speech spectrograms as image-like inputs and applies a region-based detection framework. To address data scarcity, the authors generated synthetic datasets (VCTK-Pro, VCTK-Art, AISHELL3-Pro) using articulatory and text-to-speech (TTS) simulations. The model performed well across multiple datasets and languages, and the work supports the proposed system's multilingual capability and use of synthetic data.

J. Zhang et al. [10] This paper proposed LLM-Dys, a method for generating large-scale dysfluent speech datasets using Large Language Models (LLMs). An LLM simulates realistic dysfluency patterns and produces labeled dysfluent text, which the VITS model converts into synthetic audio. Unlike rule-based methods, the generated speech keeps more natural prosody and contextual variation. The dataset covers 11 categories of dysfluencies at both word and phoneme levels, enabling fine-grained analysis and model training. The resulting models performed better on detection tasks, which backs the proposed system's use of LLMs and TTS for data augmentation.

S. Kim and A. Kumar [11] FluentNet, a hybrid CNN-LSTM architecture for end-to-end speech disfluency detection, was developed in this paper. CNN layers extract short-term spectral features from mel-spectrograms, and LSTM layers model temporal continuity, which suits recurring stutter patterns. Trained and validated on SEP-28k, the model passed 91% classification accuracy. It also ran in real time in speech therapy, giving users visual dysfluency feedback within a feedback loop. End-to-end models that skip handcrafted features reduce bias and generalize across speakers and environments, and the architecture informs the multi-model CNN approach in the proposed system. The recurring temporal recurrent connections ensure that transient acoustic disruptions are tracked continuously across adjacent speech segments without boundary truncation.

R. Ahmed and J. Park [12] This study developed a multilingual speech dysfluency detection framework that identifies stuttering events across English, Korean, and Japanese datasets. Built on an encoder-decoder transformer architecture similar to BERT, the model captures contextual relationships in speech sequences. Trained on combined multilingual datasets, it showed high adaptability to linguistic variation, achieving an F1-score of 95.2% in English and over 93% in non-English corpora. Attention visualization showed which segments of the input most influenced the detection decision. The work supports the proposed system's cross-language generalization and interpretability in dysfluency detection.

F. Rahimi and D. Torres [13] This research presented an exploration of large language models (LLMs) and transformer-based architectures for speech dysfluency analysis. Speech features are embedded as tokenized sequences so the model can detect contextual disruptions in spoken language. The LLM-based approach outperformed CNN and RNN models in both recall and precision, particularly for subtle dysfluency types like interjections and soft blocks. Pretraining on large general speech datasets also helped on smaller stutter-specific corpora, and attention visualization made the predictions transparent. The findings point to combining CNN-based acoustic analysis with language-level context in the proposed system.

V. Uloza et al. [14] An AI-based algorithm for assessing pathological speech, specifically substitution voicing, was developed in this paper using deep neural networks. CNNs and principal component analysis (PCA) perform the classification, with preprocessing such as noise removal and normalization giving the networks consistent input. Classification accuracy exceeded 93%, confirming that AI can detect subtle speech impairments. Though the study targets substitution voicing rather than stuttering, it offers useful lessons for speech pathology systems built on spectral analysis, which is the basis for the proposed system's use of mel-spectrograms and CNN feature extraction.

H. Müller and C. Lee [15] This study analyzed the neural bases of speech production in people who stutter through a meta-analysis of fMRI and EEG studies. Identified regions include the inferior frontal gyrus and basal ganglia, which show atypical activation during speech production. These findings provide biological grounding for AI-based stutter detection, since the acoustic signatures such systems rely on reflect underlying differences in neural control. The study proposes no computational model, but it links the speech irregularities the proposed system detects to their neurological causes, which is useful for feature selection and interpretation.

A. Baevski et al. [16] wav2vec 2.0 was introduced in this paper as a self-supervised framework for learning speech representations directly from raw audio waveforms. A convolutional feature encoder extracts latent representations that a transformer encoder then contextualizes. Pretrained on 960 hours of unlabeled LibriSpeech data and fine-tuned with a CTC loss, the model performed strongly on phoneme recognition, speaker identification, and emotion recognition benchmarks. This is the foundational speech representation model in the proposed system, making feature extraction from stuttered speech possible without large labeled datasets.

P. Khanna et al. [17] The study presented StuD, a multimodal stuttering detection system that combines acoustic features from Wav2Vec 2.0 and HuBERT with linguistic features from Llama-2, enhanced by Retrieval-Augmented Generation for adaptive classification. A fusion strategy weights the acoustic and linguistic embeddings by input quality. The RAG component retrieves similar historical cases to improve classification of rare stuttering patterns. Evaluated on SEP-28k and FluencyBank, the system performed strongly across all stuttering event types. This multimodal fusion approach supports the proposed system's goal of combining audio and text features for comprehensive stutter analysis.

C. Lea and V. Mitra [18] SEP-28K, a dataset of over 28,000 speech clips drawn from stuttering support group podcasts and annotated for five stuttering event types (blocks, prolongations, repetitions, interjections, and revisions), was introduced in this paper. The clips carry both crowdsourced annotations from non-expert listeners and expert annotations from speech-language pathologists. The authors report that scaling annotation from 10k to 28k clips improved F1 by 28% and 24% for blocks and prolongations respectively. The dataset addresses the data scarcity problem in stuttering research and is the primary benchmark for evaluating the proposed detection system.

O. Shonibare et al. [19] This paper proposed a two-stage approach for automatic speech recognition on stuttered speech with limited labeled data. A context-aware classifier trained on small amounts of labeled stuttering data detects dysfluency regions. A modified ASR decoder then passes over the detected dysfluencies, focusing transcription on fluent segments. The method cut word error rate by 12-71% across different stuttering severity levels. The detect-then-transcribe design shows that dysfluency detection and speech processing can be decoupled, which is the premise of the proposed system's modular architecture. This pipeline validates that selective temporal gating shields subsequent downstream transcription layers from severe acoustic degradation during disfluent periods.

S. Bayerl et al. [20] The study applied fine-tuned wav2vec 2.0 to dysfluency detection in clinical stuttering therapy recordings. Acoustic embeddings are combined with a multi-task learning framework and an SVM classifier to distinguish dysfluency types in therapy speech. Evaluated on FluencyBank and the German KSoF dataset, it improved F1 by 27% over baseline methods. The multi-task approach predicts dysfluency presence and type together, letting therapists track progress over sessions. The clinical context confirms that pretrained speech models transfer to real-world stuttering assessment.

S. Bayerl et al. [21] This paper introduced a modified wav2vec 2.0 framework that treats stuttering detection as a multi-label classification problem, on the grounds that dysfluencies often occur together in natural speech. The model assigns an independent probability score to each dysfluency type, so multiple overlapping events are detected at once. It performed well on SEP-28k-Extended and generalized from English to German. The multi-label formulation fits the reality of stuttering, where a single utterance can contain blocks, prolongations, and repetitions at the same time.

R. Gong et al. [22] AS-70, the first and largest Mandarin stuttered speech dataset, was introduced in this paper, containing 70 hours of recordings from speakers at varying stuttering severity levels. The corpus covers diverse demographics (age groups, genders, and stuttering types) alongside word- and syllable-level dysfluency labels. The dataset is released as an open source resource to encourage Mandarin stuttering research. It extends stuttering data beyond English and enables the development of multilingual systems.

X. Liu et al. [23] An end-to-end stuttering detection architecture was introduced in this paper, combining Conformer blocks for local acoustic pattern extraction with BiLSTM layers for long-range temporal dependency modeling. The multi-task framework predicts dysfluency types and severity levels together. It took first place in the SLT 2024 Stuttering Detection Challenge and improved F1 by 39.8% on the AS-70 Mandarin dataset. Conformer's mix of convolution and self-attention captures both local and global speech patterns, which proved effective for the varied temporal signatures of different stuttering events.

A. Batra et al. [24] Boli, a multilingual Indian language dataset for understanding stuttering including both read and spontaneous speech, was introduced in this paper. The corpus captures real-world stuttering patterns from Indian speakers across five stuttering types. The dataset includes demographic metadata and self-reported stuttering severity ratings. It addresses the gap in non-English stuttering datasets, particularly for South Asian

languages where stuttering prevalence and manifestation patterns differ from Western populations.

A. R. Valente et al. [25] This study introduced an enhanced version of the FluencyBank dataset with detailed clinical annotations provided by expert speech-language pathologists. The annotations cover audiovisual cues and secondary stuttering behaviors such as facial tension and eye blinking, plus physiological tension indicators, beyond the usual acoustic dysfluency labels. The multi-dimensional scheme captures more of the clinical picture of stuttering severity. Such expert annotations make it possible to train detection models that account for the full stuttering experience rather than acoustic events alone.

T. Grósz et al. [26] This paper presented a wav2vec2-based paralinguistic analysis system that jointly recognizes vocalized emotions and stuttering events from speech. Self-supervised speech embeddings feed task-specific classification heads for both tasks. The system reached 62.1% unweighted average recall on the Stuttering Sub-Challenge benchmark. The results connect emotional state and speech disfluency, and suggest that modeling the two together improves detection through shared representations.

J. Tang et al. [27] This paper presented comprehensive speech annotation guidelines developed in collaboration with people who stutter, standardizing how stuttering events are labeled in speech datasets. The guidelines resolve the hard cases in annotating ambiguous dysfluencies with clear categorical definitions and decision trees. They also cover annotator training, inter-annotator agreement measurement, and handling edge cases. Community-informed guidelines raise annotation quality and reproducibility across stuttering research, which directly helps dataset construction and model evaluation.

A. Romana et al. [28] The study introduced an updated version of the FluencyBank dataset with word-level timestamps and refined disfluency annotations. Updated transcripts carry granular disfluency labels and word timing for each clip, letting speech processing models be measured on disfluent input. The resource supports word-level dysfluency localization in stuttering research, which is exactly what the proposed system needs to map events to the words where they occur.

L. Nie et al. [29] MMSD-Net, the first multimodal neural framework for stuttering detection combining audio and visual signals through transformer-based cross-modal fusion, was introduced in this paper. The model processes speech embeddings alongside facial expression and lip movement features to capture both the acoustic and visual

manifestations of stuttering. It improved F1 by 2-17% over single-modality approaches on benchmark datasets. Visual information clearly helps, especially for events like blocks and prolongations that have distinct visual signatures.

R. P. Buzzeti et al. [30] This paper proposed a two-stage hybrid approach for detecting and classifying stuttering-related disfluencies in Brazilian Portuguese. The first stage identifies potential disfluency regions from linguistic rules, and the second performs severity assessment. Portuguese stuttering differs from English in syllable structure and how dysfluencies manifest, and the method addresses those language-specific patterns. It extends stuttering detection to an underrepresented language and shows that hybrid approaches combining linguistic rules with data-driven classification work.

The studies surveyed above range from acoustic classifiers and self-supervised speech representations to multimodal fusion and synthetic data generation. A consolidated comparison of this body of work is presented in Table 1.1, which lists each study along with the features it exploits, its principal strengths, and its limitations. Taken together, the works trace a clear shift from hand-crafted acoustic features toward large pre-trained speech models, and they point to the gaps in multilingual coverage, localization, and clinical reliability that this project sets out to address.

Table 1.1: Summary of Literature Survey

Sl. No	Author	Title	Features	Pros	Cons
1	Pierre Arbajian et al. [1]	Effect of speech segment samples selection in stutter block detection and remediation	Analyzed different speech segment lengths and sample selection methods for accurate stutter block detection using acoustic classifiers.	Improves detection precision through better segment selection.	Sensitive to segmentation configuration.
2	Vikramjit Mitra et al. [2]	Analysis and Tuning of a Voice Assistant System for Dysfluent Speech	Modified ASR decoding parameters by increasing word insertion penalty and reducing acoustic model influence to suppress repetition errors	Enhances intended speech recognition for dysfluent users.	Focuses on recognition rather than dysfluency classification.

Sl. No	Author	Title	Features	Pros	Cons
3	Payal Mohapatra et al. [3]	Speech Disfluency Detection with Contextual Representation and Data Distillation	Developed DisfluencyNet using contextual embeddings and distilled high-confidence samples for efficient low-resource training	Achieves strong results with reduced training data	Depends on carefully filtered annotations
4	Jiajun Liu et al. [4]	Automatic Speech Disfluency Detection Using Wav2Vec 2.0 for Different Languages	Applied Wav2Vec 2.0 contextual embeddings for multilingual disfluency detection across variable speech durations.	Supports multilingual detection with strong contextual learning.	High resource requirements and limited availability of multilingual stuttering datasets.
5	Amrit Romana et al. [5]	Automatic Disfluency Detection from Untranscribed Speech	Built multimodal Bi-LSTM combining WavLM acoustic signals with BERT linguistic features from Whisper transcripts.	Detects disfluencies without manual transcription.	Multimodal design increases system complexity.
6	Dominik Wagner et al. [6]	Large Language Models for Dysfluency Detection in Stuttered Speech	Combined Wave2Vec 2.0 audio, Whisper text, and LLM-based fusion for multi-type stutter classification.	Improves multi-class detection accuracy.	Requires heavy computational resources.
7	Shakeel A. Sheikh et al. [7]	Advancing Stutter Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning	Proposed multi-contextual StutterNet with augmentation and weighted loss for robust stuttering detection.	Addresses data scarcity and class imbalance.	Synthetic augmentation may reduce realism.
8	X. Zhou et al. [8]	YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection	Adapted YOLO on speech spectrograms for simultaneous dysfluency type prediction and temporal localization.	Enables precise real-time stutter localization.	Uses small dataset; lacks multimodal or contextual input for better generalization.
9	Xuanru Zhou et al. [9]	Stutter-Solver: End-to-End Multi-Lin-	Developed multilingual YOLO-based dysfluency detector	Expands multilingual coverage with SOTA accuracy.	Synthetic speech may not fully match real speech.

Sl. No	Author	Title	Features	Pros	Cons
		gual Dysfluency Detection	using synthetic articulatory and TTS-generated datasets.		
10	Jinning Zhang et al. [10]	Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection	Proposed LLM-Dys framework using LLM-generated dysfluent text and VITS TTS for scalable corpus creation.	Generates diverse large-scale dysfluency datasets.	Synthetic prosody may limit robustness.
11	S. Kim & A. Kumar [11]	FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning	CNN-LSTM hybrid model analyzing temporal and spectral dependencies in speech signals using SEP-28k dataset.	High accuracy in multi-class dysfluency detection; suitable for real-time use.	Model complexity increases training time and requires GPU-based systems.
12	R. Ahmed & J Park [12]	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Transformer-based multilingual model for detecting stuttering across multiple languages using contextual embeddings.	Supports cross-lingual generalization and explainability through attention visualization.	High resource requirements and limited availability of multilingual stuttering datasets.
13	F. Rahimi & D. Torres [13]	Large Language Models for Dysfluency Detection in Stuttered Speech	Used transformer-based large language models (LLMs) to capture context disruptions in stuttered speech	Outperforms CNN and RNN baselines on subtle dysfluency types.	Requires heavy computational resources.
14	V. Uloza et al. [14]	An AI-Based Algorithm for the Assessment of Substitution Voicing	CNN and PCA combine feature extraction and classification of pathological voice disorders.	Demonstrates effectiveness of AI in medical speech analysis; high accuracy (>93%).	Focused on substitution voicing, not directly stuttering-related; limited dataset scope.
15	H Muller & C. Lee [15]	Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis	Analyzed fMRI/EEG studies to identify brain regions linked to speech dysfluency.	Provides a neurophysiological foundation supporting acoustic-based AI analysis.	Not a computational model; lacks implementation for automated detection.
16	A. Baevski et al. [16]	wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations	Self-supervised pre-training of speech representations using masked prediction with trans-	Provides foundational speech embeddings used by many downstream	Requires large unlabeled data for pre-training; not stuttering-specific.

Sl. No	Author	Title	Features	Pros	Cons
			former and convolutional encoder on raw audio.	stuttering detection models.	
17	P. Khanna et al. [17]	StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies	Combined Wav2Vec 2.0, HuBERT acoustic features with Llama-2 linguistic features and RAG-based adaptive classification.	Achieves SOTA on SEP-28k and FluencyBank with adaptive retrieval.	Heavy computational requirements for LLM and RAG components.
18	C. Lea and V. Mitra [18]	SEP-28K: A Dataset for Stuttering Event Detection from Podcasts	Large-scale dataset with 28k+ clips annotated for 5 stuttering event types from podcast recordings.	Enables large-scale training with 28%/24% F1 gains from data scaling.	Podcast speech may not generalize to clinical or spontaneous settings.
19	O. Shonibare et al. [19]	Enhancing ASR for Stuttered Speech with Limited Data using Detect and Pass	Two-stage approach: context-aware dysfluency detection followed by ASR that passes over detected events.	Achieves 12-71% WER reduction with minimal labeled data.	Relies on accurate first-stage detection; cascaded errors possible.
20	S. Bayerl et al. [20]	Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0	Fine-tuned wav2vec 2.0 with multi-task learning and SVM for therapy speech analysis.	27% F1 improvement on clinical therapy recordings.	Limited to therapy contexts; may not generalize to casual speech.
21	S. Bayerl et al. [21]	Dysfluencies Seldom Come Alone - Detection as a Multi-Label Problem	Modified wav2vec 2.0 for simultaneous multi-label detection of co-occurring dysfluencies.	SOTA on SEP-28k-Extended with cross-language generalization.	Multi-label training increases model complexity and annotation requirements.
22	R. Gong et al. [22]	AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition	First large-scale Mandarin stuttered speech dataset with verbatim transcriptions and detailed annotations.	Enables Mandarin stuttering research with 70h of diverse speaker data.	Limited to Mandarin; annotation process labor-intensive.
23	X. Liu et al. [23]	An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM	Conformer blocks with BiLSTM temporal modeling for multi-task dysfluency and severity prediction.	1st place SLT 2024 Challenge; 39.8% F1 improvement on AS-70.	Conformer architecture requires significant GPU memory and training time.

Sl. No	Author	Title	Features	Pros	Cons
24	A. Batra et al. [24]	Boli: A Dataset for Understanding Stuttering Experience	Multi-lingual Indian language dataset with read and spontaneous speech across 5 stutter types.	Captures real-world Indian stuttering patterns across multiple languages.	Limited annotations; may need expert review for clinical applications.
25	A. R. Valente et al. [25]	Clinical Annotations for Automatic Stuttering Severity Assessment	Enhanced FluencyBank with expert clinician annotations including audiovisual and behavioral cues.	Provides clinical-grade multi-dimensional annotations for severity assessment.	Expert annotation is expensive and time-consuming to produce.
26	T. Grósz et al. [26]	Wav2vec2-based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering	Joint wav2vec2 framework for emotion recognition and stuttering detection from speech embeddings.	Demonstrates emotion-disfluency interconnection with 62.1% UAR.	Joint modeling may introduce task interference; limited to paralinguistic analysis.
27	J. Tang et al. [27]	Speech Annotation Guidelines with People Who Stutter	Standardized annotation protocols developed collaboratively with PWS for consistent dysfluency labeling.	Improves annotation quality and reproducibility across stuttering research.	Guidelines adoption requires community-wide coordination and training.
28	A. Romana et al. [28]	FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition	Updated FluencyBank with word-level timestamps and refined disfluency annotations for precise temporal alignment.	Enables word-level dysfluency localization and more accurate model evaluation.	Limited to English stuttering patterns; requires updated annotation protocols.
29	L. Nie et al. [29]	MMSD-Net: Towards Multi-modal Stuttering Detection	First multi-modal neural framework combining audio and visual signals through transformer-based cross-modal fusion.	2-17% F1 improvement over uni-modal approaches; captures visual stuttering cues.	Requires video input; increased computational complexity and data collection overhead.
30	R. P. Buzzeti et al. [30]	Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese	Two-stage hybrid approach combining rule-based detection with ML classifica-	Extends stuttering detection to underrepresented languages; clinically applicable.	Language-specific rules may not transfer to other languages without modification.

Sl. No	Author	Title	Features	Pros	Cons
			tion for Portuguese stuttering.		

1.4 PROBLEM STATEMENT

Stuttering assessment in clinical practice still depends heavily on manual observation, where a speech-language pathologist listens to recordings and marks dysfluencies by hand. This process is slow, subjective, and difficult to reproduce, and the disagreement between raters makes it hard to track a patient's progress reliably. Existing automated tools mostly stop at binary detection, reporting that a recording contains stuttering without indicating which type occurred or where in the audio it occurred, which limits their usefulness for therapy planning.

The problem addressed in this project is therefore to build a single pipeline that takes an unconstrained speech recording and produces a structured analysis: whether stuttering is present, which of the five dysfluency types are involved, and the exact time regions where each event occurs. The system must also remain usable without large annotated corpora from the target speaker and must run fast enough to support interactive assessment rather than offline batch processing.

1.5 OBJECTIVES

The project is organized around four objectives, each contributing one stage of the final pipeline.

1. **Speech preprocessing:** clean and standardize the input audio so that every downstream model receives consistent data. This covers resampling to a fixed rate, DC offset removal, peak normalization, silence trimming, fixed-length padding, and augmentation such as noise injection and time or pitch shifting, which together reduce the effect of varying recording conditions and speaker loudness.
2. **Stutter detection:** determine whether a given speech sample contains dysfluent speech at all. This objective produces the binary signal that gates the rest of the analysis, so it is trained to be robust across speakers, languages, and background noise rather than tuned only for clean studio recordings.
3. **Stutter classification:** identify which dysfluency types are present among prolongation, block, sound repetition, word repetition, and interjection. Because these events often co-occur, each type is modelled as an independent decision with its own confidence score instead of forcing a single mutually exclusive label.

4. **Stutter localization:** find the time regions in the waveform where the detected events occur and align them with the corresponding words in the transcript. This objective turns a clip-level prediction into actionable timestamps, allowing clinicians to jump directly to the affected segment and compare the detected events against the spoken content.

These objectives are ordered so that each one builds on the output of the previous stage. Preprocessing produces a normalized waveform and spectrogram representation that both the detection and classification models consume, detection narrows the analysis to samples that actually contain dysfluencies, classification assigns type-level scores to those samples, and localization maps the detected events back onto the time axis and the transcript.

CHAPTER 2

REQUIREMENT SPECIFICATION AND ANALYSIS

2.1 INTRODUCTION

The development of a deep learning-based framework for stuttering classification and localization requires a clear understanding of both clinical and technical aspects. Before implementation, it is important to define specific functional and non-functional requirements, system components, and workflow to ensure accuracy and clinical reliability. The main goal is to transform the manual process of speech-language assessment into an efficient, automated computational model that supports early and objective dysfluency detection. The system should effectively process audio recordings, extract key features, and classify them into distinct stuttering types while localizing dysfluency events within the signal. It must address challenges like time-consuming manual analysis and inter-observer variation, emphasizing automation, precision, and interpretability. Technically, the framework integrates Wav2Vec 2.0 feature extraction with Wav2Vec2 frame-level localization and Whisper transcription for robust detection and localization. In addition, the system should operate efficiently on standard hardware, support scalability for future datasets, and offer a user-friendly interface for clinicians and end users. Data security and confidentiality are essential to maintain ethical and legal standards. Overall, the requirement specification and analysis phase ensures that the framework is both technically strong and clinically valuable, bridging the gap between healthcare needs and AI-driven solutions.

2.2 FUNCTIONAL REQUIREMENTS

The functional requirements describe the key operations that the system must perform to support automated detection and classification of stuttering events using deep learning models. The application allows users, such as clinicians, speech-language pathologists, or individuals who stutter, to upload a speech recording or capture audio directly through the interface in common formats such as WAV, MP3, FLAC, and M4A. Once the audio is provided, the system processes the signal and classifies it into five dysfluency types: prolongation, block, sound repetition, word repetition, and interjection. The classification result is displayed on the interface along with a confidence score for each detected class, helping users understand how reliable every prediction is. The system also localizes dysfluency events within the audio and returns their exact temporal positions as time-

stamps, with visualization through waveforms, spectrograms, and dysfluency overlays that highlight the regions that influenced the model's decision, improving interpretability. Additionally, users can download the detailed analysis report for record-keeping. The framework ensures that uploaded audio and prediction results are stored securely for future reference.

2.2.1 User Requirements

User requirements specify the functionalities and services that the end users should be able to access and utilize. These typically reflect how users will interact with the system. The proposed system is primarily designed for clinicians, speech-language pathologists, researchers, and individuals who stutter involved in the assessment and monitoring of speech fluency. The users of this system require an intuitive, reliable, and efficient platform that can process speech recordings and provide accurate diagnostic insights with minimal manual effort. The users expect the interface to be simple, interactive, and easy to navigate, allowing them to view classification results, visual interpretations, and confidence levels without technical complexity.

For a speech analysis platform, general user requirements include:

- The user should be able to upload or record a speech sample for analysis.
- The user should be able to access real-time classification results.
- The system should provide detailed analysis reports.

2.2.2 System Requirements

System requirements define the essential capabilities of software and hardware needed to support system functionality. The proposed stuttering detection system requires components that can efficiently handle the computational demands of deep learning-based speech analysis while delivering the core functional operations described above. Hardware guidelines appear in the hardware requirements section later in this chapter.

These requirements include:

- The system should support secure audio upload and classification.
- Users should be able to store and retrieve previous analysis results.
- The system should support proper storage for managing large speech datasets.

2.3 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements define the overall quality, performance, and usability standards that ensure the system operates efficiently and reliably. The key requirement is

performance efficiency, as the models handle deep learning and audio processing tasks that must deliver fast and accurate results with minimal delay. GPU acceleration and optimized memory management help achieve smooth processing of speech recordings. Reliability is equally important: the system should maintain consistent accuracy across datasets and handle interruptions through fault tolerance. It must also be scalable, allowing expansion for larger datasets or additional dysfluency types without compromising performance. In terms of usability, the interface should be simple, intuitive, and accessible to both technical and non-technical users for easy audio upload, analysis, and visualization. Maintainability should be ensured through modular, well-structured code that supports easy updates and retraining. Finally, portability and availability ensure the system runs smoothly across different platforms with high uptime and regular backups for uninterrupted access.

2.3.1 Performance

- The system should process and analyse a speech sample within 3-5 seconds.
- The classification results should minimize false positives and false negatives.
- Performance metrics should be monitored regularly.
- The system should handle processing of audio uploads without significant delay.

2.3.2 Scalability

- The system should efficiently handle increasing speech datasets.
- It should be scalable for integration across multiple clinics and research centres.
- The system should be adaptable for improvements in deep learning models.
- The system should be capable of handling a higher volume of concurrent users.

2.3.3 Availability

- The system should remain accessible to users at all times, ensuring minimal downtime during clinical use.
- System components are designed to be fault-tolerant to reduce the impact of unexpected runtime failures.
- The platform should support uninterrupted service even during updates, model improvements, or cache refreshes.
- Automated health-check endpoints and graceful recovery mechanisms maintain service continuity during high-demand analysis periods.

2.3.4 Usability

- The user interface should be user-friendly for clinicians, speech-language pathologists, and non-technical users.
- The system should provide clear navigation for audio uploads and results.
- The visualization outputs shall help users easily understand the model's predictions for the analysed speech.

2.4 Software Requirements

The software requirements are essential for developing, training, validating, and deploying deep learning models aimed at detecting stuttering events from speech audio. The system must support various tools and libraries that enable efficient model development, audio processing, and performance evaluation.

- **Operating System:** the proposed system is developed and executed on the Linux and Windows platforms. It is fully compatible with Windows 10 and above and common Linux distributions, offering stability, wide software support, and seamless integration with deep learning frameworks and GPU drivers. This environment ensures smooth performance, easy setup, and reliable execution of the proposed models, with both the web application and the PySide6 desktop application running on either platform and training and inference supported on CPU or NVIDIA GPU.
- **Deep Learning Framework:** the proposed system is built using PyTorch as the primary deep learning framework for building and training the machine learning models. PyTorch provides the required computational efficiency, GPU support through features such as torch.compile, mixed precision, and TensorFloat-32 (TF32), and scalability for handling complex audio classification tasks effectively. The Hugging Face Transformers library supplies the pretrained speech models, Wav2Vec 2.0 for feature extraction and Whisper for timestamped transcription.
- **Libraries:** a comprehensive range of Python scientific libraries is required to support the different processing stages across the pipeline:
    - librosa: Used for audio preprocessing and manipulation, including loading, resampling, mel-spectrogram computation, and silence trimming, standardizing acoustic properties prior to neural ingestion and extracting spectral centroid features for unvoiced boundary checks.

- NumPy: Provides support for numerical operations and array handling.
- Pandas: Essential for managing and analysing structured data such as labels or metadata.
- Scikit-learn: Used for data splitting, model evaluation metrics, and traditional machine learning algorithms.

2.5 Hardware Requirements

The hardware requirements outline the minimum and recommended specifications needed to efficiently run the deep learning models used in this project. Proper hardware ensures smooth data processing, model training, and inference performance, which is critical when working with speech audio and deep learning pipelines.

- **Processor (CPU):** a multi-core processor such as Intel Core i5 (8th gen or newer) or AMD Ryzen 5 is required for audio preprocessing and pipeline execution.
- **Storage (Hard Disk):** at least 10 GB of free disk space is required to store datasets, model checkpoints, logs, and cache files; an SSD is strongly recommended.
- **Memory (RAM):** a minimum of 8 GB RAM is necessary, while 16 GB or more is recommended for efficient dataset loading, augmentation, and model training.
- **Graphics Processing Unit (GPU):** an NVIDIA GPU with CUDA support (RTX 2060 or higher) is recommended to accelerate training and multi-label inference.
- **Audio Input Device:** a standard functional microphone is required for recording speech directly within the application.

The minimum configuration is sufficient for inference on short clips, while training or fine-tuning the larger pretrained backbones benefits from the recommended configuration; classification-only mode runs comfortably on a standard CPU.

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION

System design is an essential stage in software development that focuses on defining the overall structure and functionality of a system. It translates user and technical requirements into a detailed framework that outlines how different components, modules, and interfaces will interact. The process involves creating both high-level architecture, which defines the system's major elements and their interconnections, and low-level design, which specifies detailed components such as data structures, algorithms, and communication flows.

The main objective of system design is to build a solution that fulfills user needs while ensuring performance, reliability, scalability, and security. It also emphasizes maintainability and flexibility so the system can adapt to future enhancements. Effective system design requires collaboration among developers, architects, and domain experts to ensure the architecture aligns with functional goals and business requirements. By carefully organizing how each part of the system operates and communicates, system design establishes a strong foundation for efficient implementation and long-term stability.

3.2 SYSTEM ARCHITECTURE AND DESIGN

The system architecture defines the overall structure and organization of the Stuttering Classification and Localization System using deep learning. It provides a comprehensive framework describing how different components interact to process speech audio, extract discriminative features, and classify utterances into normal and dysfluent categories. A well-structured architecture ensures efficient data flow, modularity, and scalability, allowing the model to deliver high diagnostic accuracy while remaining adaptable to future advancements. The architecture incorporates critical stages such as preprocessing, feature extraction, model training, and classification to ensure robust performance. Furthermore, it supports seamless integration with user interfaces for audio upload, visualization, and prediction display. Emphasis is placed on maintaining data security and patient confidentiality, ensuring ethical and reliable operation throughout the analysis workflow. Additionally, the architecture allows for performance optimization, enabling fast and accurate stutter detection even when deployed on large speech datasets, while maintaining low latency and high diagnostic throughput across clinical workstations.

The proposed deep learning architecture for the automatic classification and localization of stuttering using speech audio is illustrated in Figure 3.1. The framework integrates two model pathways, five independent Wav2Vec 2.0 binary classifiers for classification and a Wav2Vec2 frame-level pipeline for localization, to improve feature extraction, multi-perspective learning, and classification robustness. The complete workflow consists of five major phases: Input, Pre-processing, Feature Extraction, Classification, and Output. This dual-model structure enables richer representation learning and enhances overall detection accuracy.

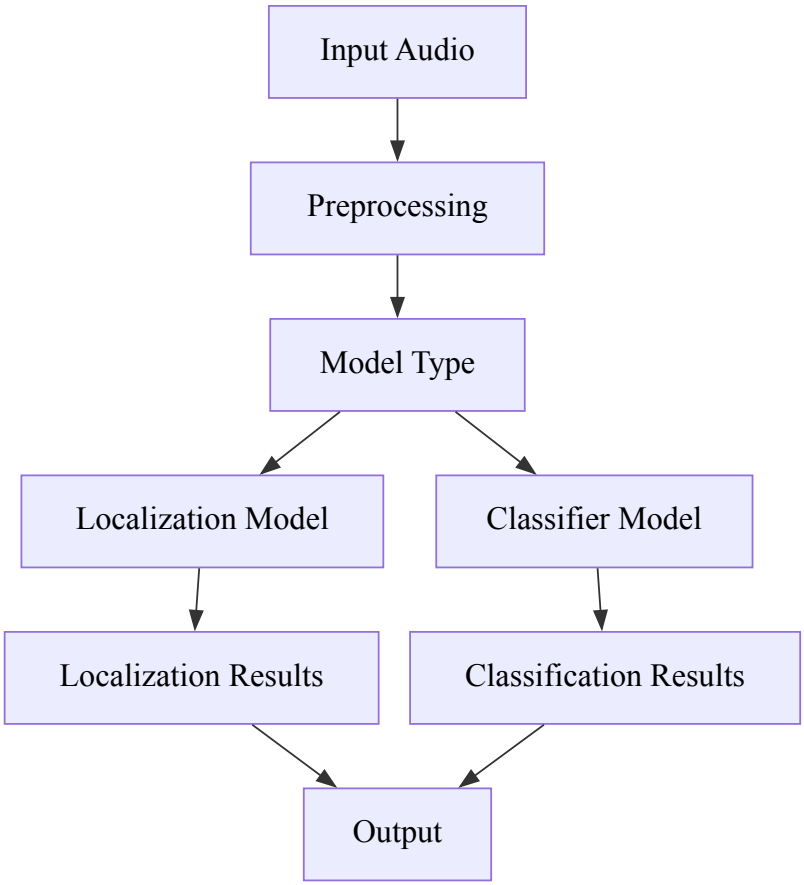


Figure 3.1: System Architecture

Speech Audio Input: The system begins with the collection of speech audio samples, which serve as the primary dataset for stutter detection. Audio enters the system when the user records directly or uploads a pre-recorded file through the web interface or desktop application. The web app captures audio with MediaRecorder APIs; the desktop app uses sounddevice. Each sample may contain one or more dysfluencies, including blocks, prolongations, word repetitions, sound repetitions, and interjections. Ensuring adequate recording quality is crucial, as variations in microphone hardware, background noise, and speaking rate can affect model reliability. The dataset includes recordings from Project Boli, SEP-28K, and UCLASS, and the ingestion pipeline resolves the English material of

all three sources. This stage establishes the foundation of the diagnostic system by providing high-quality data for subsequent processing.

Preprocessing: Before being fed into the deep learning model, speech audio undergoes preprocessing to enhance clarity, reduce noise, and ensure consistency across samples. The audio is sent to the backend and converted to 16 kHz mono WAV with FFmpeg so every clip is handled the same way. The preprocessing pipeline also includes DC offset removal, peak normalization, and silence trimming to clean the signal and remove unwanted artifacts. Fixed-length padding truncates or pads each clip to 48,000 samples, three seconds at 16 kHz, so all inputs have the same length. This step ensures that the input audio is standardized, optimized, and ready for accurate feature extraction and classification by the deep learning architecture.

Wav2Vec 2.0: The first part of the detection framework is Wav2Vec 2.0, a deep self-supervised model pre-trained on large amounts of unlabeled speech. Wav2Vec 2.0 quantizes the raw waveform into contextualized speech representations, learning robust acoustic features without requiring manually labeled data. This structure helps the model retain important information and capture fine-grained phonetic and prosodic patterns, such as broken phonation, repetitions, and elongated sounds, which are key indicators of dysfluent speech. Since it is pre-trained on large speech corpora, the model already understands general speech features, which makes it easier to adapt to stutter detection with limited labeled data. Its ability to extract multi-level, contextual features makes it a powerful foundation for detecting stutter events with high precision.

Detection and Localization Pipelines: To achieve higher accuracy, the proposed system uses a hybrid detection framework that combines classification and localization. After preprocessing, the audio runs through three parallel pipelines: classification, transcription, and localization. The classification pipeline extracts speech representations with Wav2Vec 2.0 and passes them through five independent binary classifiers, each trained for a distinct dysfluency type. The localization pipeline identifies dysfluency regions at the frame level using Wav2Vec2 frame-level feature extraction, so it can mark the precise segments of speech where dysfluencies occur. By integrating both approaches, the system captures fine acoustic details and broader prosodic structures in the speech signal, ensuring robust results even when recordings vary in noise, clarity, or speaking style.

Classification: Once the feature extraction is complete, the Wav2Vec 2.0 representations are passed to the classification stage. Here, five independent binary classifiers assign the presence probability of each dysfluency type, using decision boundaries learned during training to separate fluent and dysfluent speech. Sigmoid outputs generate per-class prob-

abilities, while binary cross-entropy loss ensures efficient optimization during training. Their outputs are aggregated into a multi-label result that reports the presence probability of each dysfluency type and summarizes the detected classes and the primary dysfluency. A multitask shared-backbone variant was also trained for comparison, and its outputs are combined with localization at prediction time: the combiner labels each localized region with per-class saliency scores and fuses the two views into a single annotated result. This stage translates complex audio data into clear diagnostic outcomes that can assist speech-language pathologists in identifying stutter events.

Transcription and Alignment: The transcription pipeline uses the Whisper Automatic Speech Recognition (ASR) model to produce a timestamped transcript, with support for English, Kannada, and Hindi. A Connectionist Temporal Classification (CTC) based time alignment step ties the pipelines together, mapping the detected dysfluency regions to specific words or syllables in the transcript. Language-specific adapters keep the alignment accurate for English, Kannada, and Hindi.

Output: In the final stage, the system presents its predictions through a user-friendly interface. The output typically shows the detected dysfluency types along with confidence scores that indicate how certain the model is about each decision, and a summary of the primary dysfluency. To make the results more interpretable, the outputs surface as waveform overlays, spectrogram visualizations, timestamped transcripts, and a detailed clinical-style report. The outputs are designed to be simple, clear, and clinically relevant, making them suitable for practical use in clinical or self-assessment settings. By providing accurate and interpretable results, the system supports early assessment and can play a valuable role in improving outcomes for people who stutter.

Both the web and desktop applications load trained models through the centralized model registry. The registry reads from a registry module (the model/registry package) and a configuration file (registry.json), so checkpoints can be updated or replaced without changing application code, which keeps the system flexible and easy to maintain.

3.3 FLOWCHART

The data flow design follows the audio from input acquisition to final output.

The process begins with the input audio, recorded or uploaded by the user. The audio is converted to 16 kHz mono WAV with FFmpeg, then padded or truncated to 48,000 samples, three seconds at 16 kHz, so all inputs have the same length. DC offset removal and peak amplitude scaling are applied concurrently to prevent spurious baseline drifts and signal clipping from distorting downstream representations. This rigorous input condi-

tioning guarantees that background acoustic variances do not introduce artificial spectral artifacts into the neural pipelines.

After normalization, the audio runs through three parallel pipelines: classification, transcription, and localization. Classification produces dysfluency probabilities, transcription a timestamped transcript, and localization frame-level dysfluency regions.

A CTC-based alignment step combines these outputs and maps the detected dysfluencies to specific words or syllables. The final data is turned into per-word annotations, shown in waveform and spectrogram displays, and written into the analysis report.

The system also defines a structured data flow for training. Training uses three datasets: Project Boli (sourced via Git repositories), SEP-28K (approximately 28,000 audio clips), and UCLASS (both obtained via Kaggle). Each dataset goes through its own normalization functions.

The processed datasets merge into a unified format: a combined_labels.csv file with multi-label binary annotations, plus individual interval CSV files for each audio clip. The merged dataset is split into training, validation, and testing subsets with an 80:10:10 split, with the training subset further re-split 80/20 during model training. The subsets are organized with symbolic links for efficient access, and preprocessed audio files are cached to speed up training.

During feature extraction, the preprocessed 16 kHz audio is transformed into both raw waveform tensors for the Wav2Vec 2.0 convolutional feature encoder and 80-channel log-mel spectrograms for spectral visualization and auxiliary verification. The convolutional encoder applies temporal downsampling by a factor of 320, generating latent representations at approximately 20 ms resolution. This temporal resolution is preserved across the subsequent transformer blocks, enabling precise temporal alignment with the phonetic stream. Pre-extracted feature tensors are cached in high-speed local memory to eliminate redundant computation during repeated training and evaluation runs.

The data ingestion pipeline enforces strict quality criteria before forwarding clips to the model. Audio segments with root-mean-square energy below designated thresholds are flagged as silent, while clipping artifacts from microphone gain saturation are detected and normalized. Furthermore, the dataset harmonization procedure reconciles diverse annotation conventions across Project Boli, SEP-28K, and UCLASS, transforming heterogeneous interval markings into a standardized multi-label target matrix where each 20 ms time frame is unambiguously labeled for block, prolongation, sound repetition, word repetition, or interjection. This structured data preparation ensures that all feature tensors

fed into the deep learning networks maintain homogeneous statistical properties across both training and inference modes.

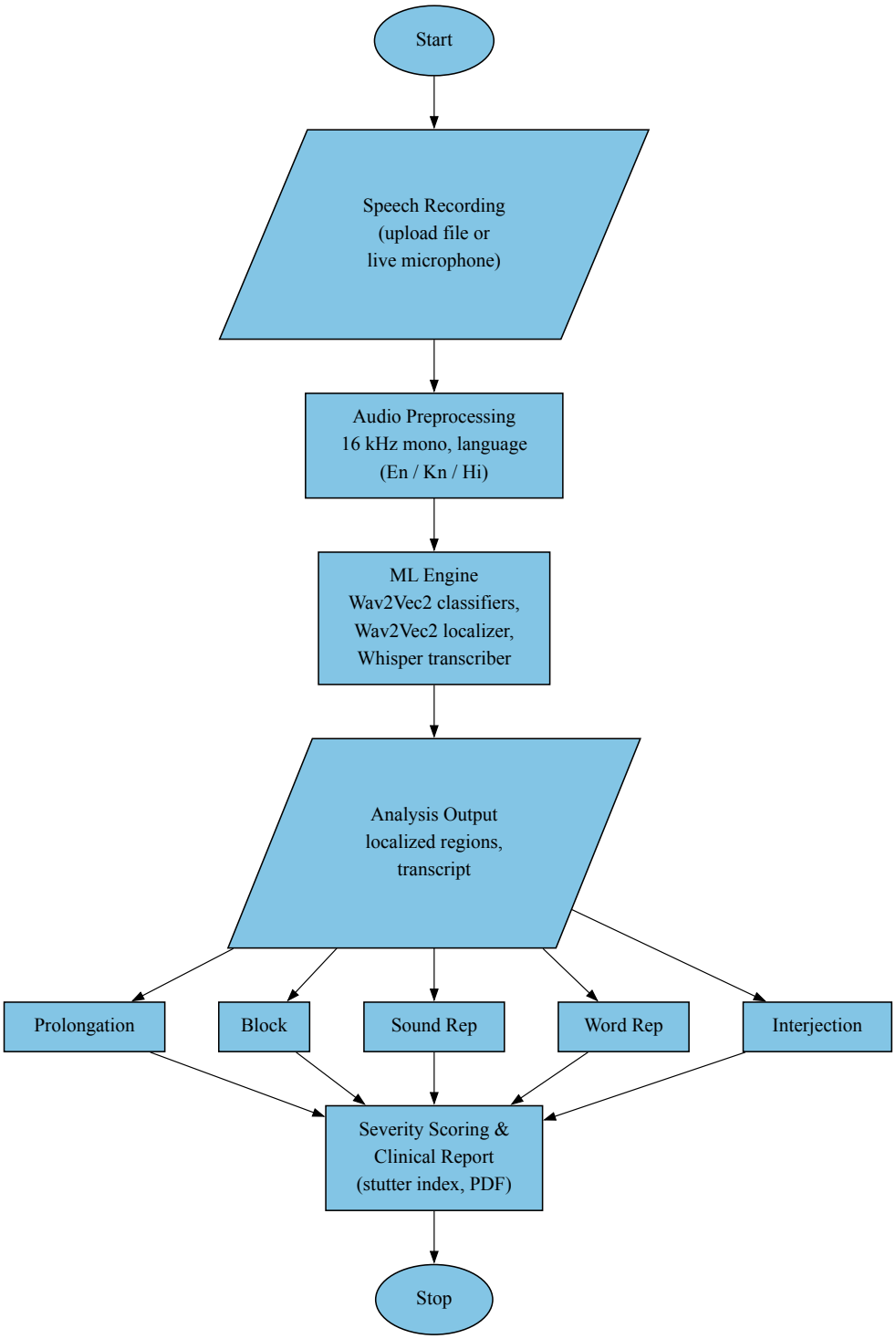


Figure 3.2: Flowchart

The flowchart shown in Figure 3.2 illustrates the workflow for stutter detection using the proposed hybrid deep learning approach. The process begins with the collection of speech audio recordings, followed by preprocessing steps such as conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding to improve signal quality and variability. The refined audio is then split into

training, validation, and testing datasets with an 80:10:10 split. Initially, the Wav2Vec 2.0 model is used to extract high-level contextual speech features. To enhance performance, a hybrid model combining five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer is developed, where the classifiers capture dysfluency-specific acoustic patterns and the localizer identifies fine-grained temporal segments. The outputs from both pipelines are fused through CTC-based alignment and passed through a final aggregation layer that categorizes the audio samples as containing blocks, prolongations, sound repetitions, word repetitions, or interjections, along with the precise localized regions. This flowchart provides a clear overview of the systematic steps involved in the proposed framework for stutter detection.

3.4 USE CASE DIAGRAM AND DESCRIPTIONS

The use case diagram shows how the user interacts with the system. The primary actor is the User, either a clinician or an individual using the application for self-assessment.

The main workflow covers recording audio, uploading pre-recorded audio, and running speech analysis in either full mode or classification-only mode. After analysis, users can view waveform visualizations, spectrograms, transcripts, and stutter detection results.

Users can also localize dysfluencies within the speech, generate analysis or clinical reports, and revisit previously recorded sessions through a history feature. Supporting actions include toggling between interface themes and accessing standardized reading passages for consistent evaluation. The diagram keeps the user in a single role so that clinicians and individuals follow the same path, from capturing speech to reviewing the outcome. Results are stored per session, which lets a user compare recordings over time and monitor progress across assessments.

The use case model delineates clear operational boundaries between clinical assessment workflows and independent self-monitoring sessions. In a clinical environment, the speech-language pathologist initiates the session, configures target dysfluency thresholds, observes live acoustic waveforms during patient speech delivery, and evaluates automated multi-label predictions against standardized diagnostic criteria. For personal self-assessment, the interface provides guided reading prompts, immediate visual feedback on fluency patterns, and historical tracking to observe progress over repeated practice sessions.

Each user interaction is captured as an atomic transaction that maintains session state consistency across model invocations. When an audio recording is finalized, the frontend prepares an encrypted multi-part request incorporating client metadata, language preferences, and analysis modality flags. The system persists the resulting classification probabilities,

frame-level interval boundaries, and timestamped transcripts to local storage, permitting offline retrieval and comparative reporting across past visits without needing to recompute intensive neural network passes.

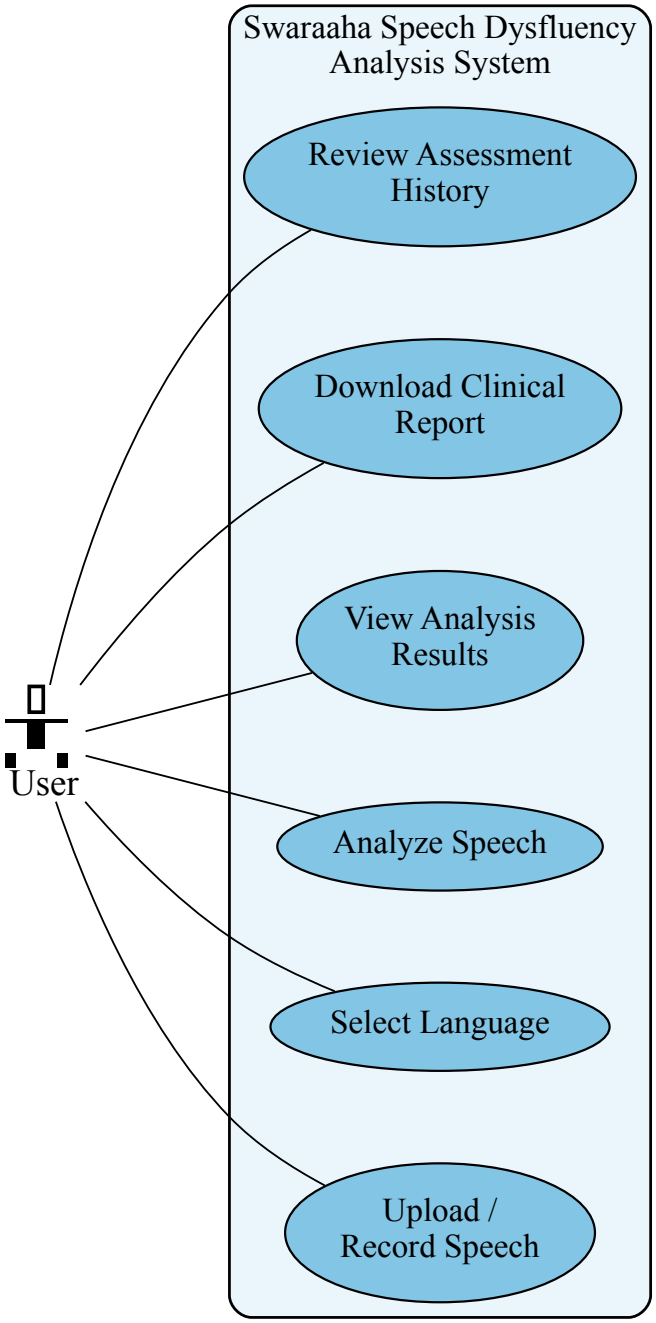


Figure 3.3: Use Case Diagram

The use case diagram shown in Figure 3.3 illustrates the interactions between the user and the stutter detection system. The user can register or log in, then either record audio through the microphone or upload a pre-recorded speech file, which is analyzed through the full or classification-only pipeline. The system displays the classification results with confidence scores, localized dysfluency regions, and a timestamped transcript, and allows the user to generate and download analysis or clinical reports. Additional use cases in-

clude viewing the session history, switching between light and dark themes, and accessing standardized reading passages for consistent assessment. This diagram provides a clear overview of the functional interactions supported by the proposed framework.

3.5 WORKFLOW DIAGRAM

A workflow diagram is a structured visual representation of a sequence of operations or tasks carried out to complete a specific process, typically used to analyse, design, or manage complex systems. It employs standardized symbols such as rectangles to denote actions, diamonds for decision points, and arrows to indicate the direction of flow, thereby offering clarity and insight into the procedural steps involved. In the context of deep learning and stutter detection, the workflow diagram illustrated here systematically maps out the process for developing a stutter classification and localization system.

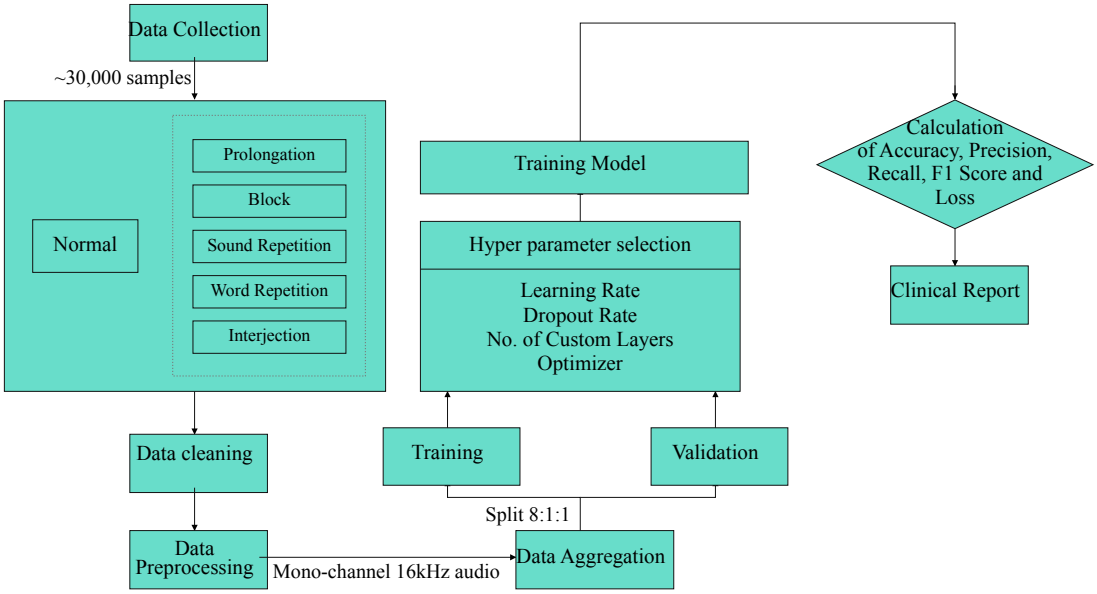


Figure 3.4: Workflow Diagram

The workflow of the proposed deep learning pipeline for stutter detection and localization is shown in Figure 3.4. The process begins with the collection of speech audio samples from three public stuttering datasets, Project Boli, SEP-28K, and UCLASS, all converted to 16 kHz mono and standardized through DC offset removal, peak normalization, silence trimming, and fixed-length padding. Data augmentation techniques such as random noise injection, time stretching, pitch shifting, temporal shifting, and amplitude scaling are applied to the waveforms, along with time and frequency masking on the spectrograms, expanding the effective dataset so the models generalize well across varied speaking patterns. The processed audio is then split into training, validation, and testing sets using an 80:10:10 ratio, with the training subset further re-split 80/20 during training. Model

development begins with fine-tuning a pre-trained Wav2Vec 2.0 backbone, followed by a hybrid architecture that combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer to capture both global dysfluency patterns and fine-grained temporal segments. Hyperparameters including learning rate, backbone freeze duration, optimizer, and the number of trainable layers are tuned to enhance performance, and the trained network is periodically validated to ensure stable learning and prevent overfitting. Finally, evaluation metrics such as precision, recall, F1-score, AUROC, and mean IoU are computed, and the best-performing model is selected for reliable automated assessment of speech dysfluencies.

3.6 SEQUENCE DIAGRAM

The sequence diagram shows how the components interact during execution. The user works through the graphical user interface (GUI), which sends a request to the backend API endpoint (/api/analyze).

The backend runs the request through a chain of services: preprocessing, classification, transcription, localization, and alignment. Each service does its part and passes its output to the next stage. When processing completes, the results go back to the frontend, where the user sees them. The system also stores the results for report generation and history management. The sequence diagram shown in Figure 3.5 illustrates the interaction flow between the user, the frontend, the backend services, and the stored results during a speech analysis session. The user initiates the analysis through the graphical user interface, which sends a request to the backend API endpoint. The backend sequentially invokes the preprocessing, classification, transcription, localization, and alignment services, each consuming the output of the previous stage. Once the processing completes, the fused results are returned to the frontend for display and saved in the local storage for reports and the session history. This diagram provides a clear overview of the message exchange that drives the proposed framework.

The sequencing logic accounts for both synchronous REST invocations and asynchronous real-time streaming interactions. When streaming live microphone audio, the client divides the incoming acoustic stream into consecutive 3-second overlapping windows with a 50% hop factor, streaming binary frames through a persistent bi-directional channel. The backend audio buffer reassembles incoming packets, tracks packet sequence identifiers, and discards redundant temporal overlap to maintain continuous temporal alignment across adjacent windows, ensuring that transmission jitter and packet latency do not disrupt the acoustic feature sequence.

To preserve deterministic pipeline behavior under diverse network and compute conditions, the communication architecture employs explicit request lifecycle states: Received, Validated, Processing, Fused, and Completed. If an intermediate processing failure occurs during transcription or localization—such as low signal-to-noise ratio or missing language dictionary tokens—the service gracefully recovers by executing fallback heuristic algorithms rather than terminating the entire pipeline. The client interface receives incremental progress signals, keeping the practitioner informed of the exact pipeline stage actively being executed.

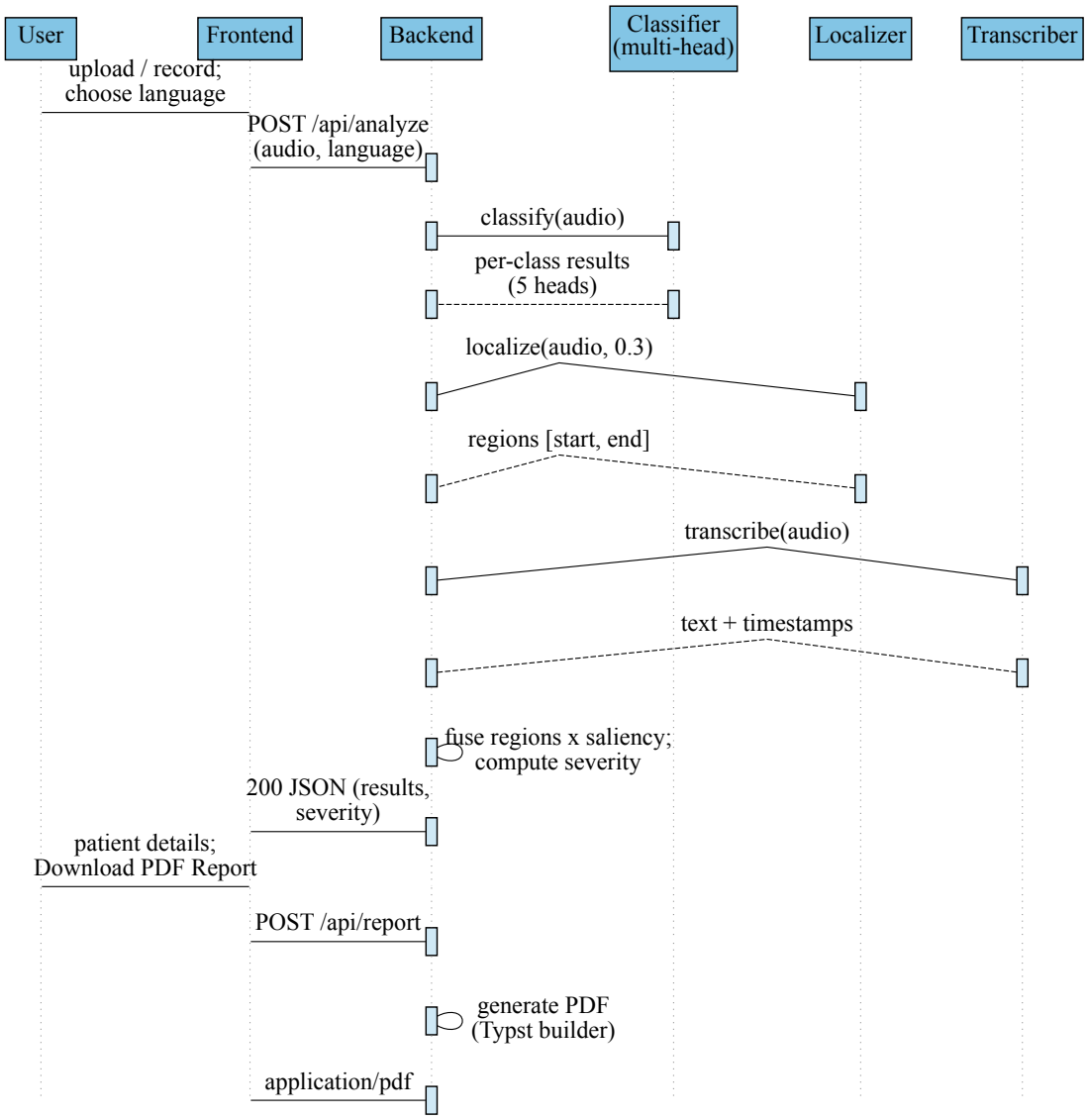


Figure 3.5: Sequence Diagram

The sequence diagram (Figure 3.5) captures the end-to-end interaction model: the GUI acts as the entry point, the backend coordinates the analysis pipeline, and the results layer handles persistence and presentation. This separation of concerns ensures modularity, traceability of requests, and consistent formatting of outputs across the web and desktop interfaces. Together, these interactions ensure that each analysis request is handled

reliably from input to report generation, with clear responsibility boundaries between components and minimal coupling between the frontend and backend services.

The request lifecycle begins when the user confirms an analysis, at which point the frontend packages the audio, the selected mode, and any optional parameters into a single request. The backend validates the payload before any model is invoked, rejecting unsupported formats or empty inputs early so that expensive inference is never wasted on malformed data. Preprocessing normalizes the waveform and produces the log-mel spectrograms and frame-level features that the downstream stages expect, acting as the single point where audio conventions are enforced. Classification then assigns probability scores to each dysfluency type, and transcription produces a timestamped transcript that the alignment stage later couples to the detected events. Localization runs in parallel with transcription where possible, since the two stages share the same preprocessed audio and do not depend on each other's output, which keeps the overall latency low. The alignment stage fuses word timings with the localized regions so that each dysfluency can be reported in the context of the spoken words around it. If any single service fails, the backend returns a structured error instead of a partial result, and the frontend surfaces the failure without discarding the user's audio, allowing the request to be retried. On success, the fused payload is stored locally so that reports and the session history can be reconstructed later without re-running the models, which also keeps repeated report generation fast and independent of the analysis services. This clear division between request handling, computation, and persistence is what allows the same backend to serve both the web and desktop interfaces while keeping behaviour identical across them.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 INTRODUCTION

The system implementation process is a multifaceted endeavor focused on developing a robust software architecture capable of effectively managing diverse user interactions, data processing tasks, and model inference operations. By employing meticulous design and coding practices, the implementation ensures that the application is scalable, reliable, and secure, capable of handling varying workloads and protecting sensitive data. Iterative testing and debugging procedures are integral components of the implementation process, allowing for the identification and resolution of any issues or bugs that may arise. This iterative approach ensures that the application delivers a seamless user experience, free from disruptions or errors. Additionally, post-deployment monitoring and maintenance mechanisms are established to continually assess system performance and address evolving requirements. This ongoing maintenance ensures that the system remains optimized and responsive to user needs, even as conditions change over time, thereby facilitating long-term success and user satisfaction.

4.2 ALGORITHM

The inference process follows a fixed sequence of steps that turns raw speech input into a complete, annotated analysis result.

- Step 1, Input Acquisition: the system acquires speech input either through real-time recording or by uploading an audio file.
- Step 2, Audio Conversion: the input audio is converted into a 16 kHz mono WAV format using FFmpeg to maintain consistency across all inputs.
- Step 3, Preprocessing: the audio is cleaned by removing the DC offset, applying peak normalization to a target amplitude, and trimming silent segments.
- Step 4, Length Normalization: the processed audio is adjusted to a fixed length of 48,000 samples (three seconds at 16 kHz) by padding or truncating as required.
- Step 5, Parallel Processing: the system processes the audio simultaneously through three independent branches:

1. Classification: Wav2Vec 2.0 embeddings are generated and passed through five binary classifiers, one per dysfluency type, and the outputs are aggregated into a multi-label result with a probability score for each dysfluency type.
2. Transcription: the Whisper model generates a timestamped transcript of the speech in English, Kannada, or Hindi.
3. Localization: the Wav2Vec2 frame-level localizer identifies the frame-level regions within the audio where dysfluencies occur.

- Step 6, Alignment: the detected dysfluency regions are mapped to the corresponding words or syllables in the transcript using CTC-based time alignment.
- Step 7, Combination and Severity: the combiner labels each localized region with per-class saliency scores from the classifier and fuses the classification and localization views, and the severity module computes a stutter index from the ratio of dysfluent speech duration to total speech duration.
- Step 8, Visualization and Report Generation: the system displays the results through waveform overlays, spectrograms, transcripts, and confidence scores, and generates a detailed clinical-style report.

4.3 IMPLEMENTATION REQUIREMENTS

This section presents the implementation requirements for the detection and localization of stuttering using a hybrid deep learning model that integrates Wav2Vec 2.0 feature extraction, five independent Wav2Vec 2.0 binary classifiers, a Wav2Vec2 frame-level localizer, and Whisper automatic speech recognition. The primary objective is to develop a robust and efficient system capable of accurately identifying speech dysfluencies and localizing them within the audio. The hybrid model leverages the strengths of both architectures: Wav2Vec 2.0 contributes self-supervised, contextual speech representations that facilitate efficient feature reuse and capture fine-grained phonetic and prosodic patterns, while Whisper introduces sequence-level transcription that enhances the model's ability to map localized dysfluency regions to specific words and syllables in the transcript. Together, these architectures form a powerful hybrid framework that improves classification performance and generalization ability. The implementation is carried out using the Python ecosystem, which provides an interactive environment for model development, experimentation, and analysis. Python serves as the primary programming language for backend development due to its versatility and support for a wide range of scientific and deep learning libraries such as PyTorch, the Hugging Face Transformers library,

and NumPy, which aid in model construction, training, and evaluation. For the frontend, React 19 and PySide6 are utilized to design a user-friendly web interface and desktop application that allow users to record or upload speech audio and view classification and localization results. All trained models are managed by a centralized model registry configured through a registry.json file that maps each task to its checkpoint and per-class thresholds, so checkpoints can be updated or replaced without changing application code. The Python ecosystem also facilitates matrix manipulation, data visualization, algorithm implementation, and seamless integration with other programming environments, making it an ideal platform for developing and testing the hybrid model for stuttering classification and localization.

4.3.1 Train the Model

The training phase is a key step in developing the proposed system for stutter classification and localization. In this phase, the speech audio dataset is first collected from Project Boli, SEP-28K, and UCLASS, and preprocessed through conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding, along with data augmentation, to enhance audio quality and reduce overfitting. Initially, the Wav2Vec 2.0 model is trained to extract high-level contextual speech features from the audio, benefiting from its self-supervised pretraining that allows efficient information flow and robust representation learning. To further improve classification accuracy, a hybrid model is developed by combining five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer. The Wav2Vec2 localizer captures fine-grained and localized temporal features at the frame level, complementing the broader segment-level feature extraction of the classifiers. The outputs from both models are then fused through CTC-based time alignment to form a comprehensive feature representation. During training, the AdamW optimizer is used to adjust learning rates adaptively and ensure stable convergence. The model is trained using mini-batches of audio, with weights updated through backpropagation based on the computed loss. Validation is performed periodically to monitor accuracy and prevent overfitting. After training, the optimized parameters and learned weights are saved, enabling the models to make efficient and accurate predictions on new speech audio without retraining.

4.3.2 Test the Model

The testing phase is carried out after completing the training process to evaluate how effectively the proposed model can detect stutter events from unseen speech audio. When the user records or uploads a speech sample through the system’s web interface or desktop application, it undergoes the same preprocessing steps used during training, including

conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding, to ensure uniformity and accurate analysis. Initially, the trained Wav2Vec 2.0 model processes the audio to extract high-level contextual speech features. To further enhance accuracy, the audio is also analyzed using the hybrid model that combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer, allowing the system to capture both segment-level dysfluency patterns and fine-grained temporal details. The extracted features from both models are fused through CTC-based time alignment to form a comprehensive representation, which is then passed through the final aggregation layer to generate the classification result as containing blocks, prolongations, sound repetitions, word repetitions, or interjections, along with the precise localized regions. The outcome is displayed on the interface in a clear and user-friendly format, with waveform overlays, spectrograms, a timestamped transcript, and confidence scores. Additionally, the model's predictions are compared with the actual ground truth labels to assess its accuracy and generalization performance. This phase confirms that the hybrid model performs effectively on new and unseen speech samples, demonstrating its potential for reliable stutter classification and localization in real-world clinical use.

4.3.3 Model Evaluation

Model evaluation is essential to assess the overall performance and reliability of the proposed system. Once the models are trained and tested, various quantitative metrics are employed to evaluate their classification performance comprehensively, including:

- Accuracy: measures the proportion of correctly predicted samples.
- Precision: evaluates how many of the predicted positive instances are truly positive.
- Recall (Sensitivity): measures how effectively the model identifies all relevant positive samples.
- F1-Score: provides a harmonic mean of precision and recall to balance both metrics.

The five Wav2Vec 2.0 binary classifiers, one per dysfluency type, are further measured with AUROC (Area Under the Receiver Operating Characteristic Curve), AUPRC (Area Under the Precision-Recall Curve), and specificity, summarized per class and as a macro average across the five dysfluency classes. The Wav2Vec2 frame-level localizer is measured with frame-level precision, recall, and F1, detection accuracy, mean Intersection

over Union (IoU) between predicted and ground-truth regions, and the false alarm rate per minute.

A confusion matrix is generated for each of the five binary classifiers to visualize the distribution of predictions, highlighting true positives, false positives, true negatives, and false negatives. This detailed evaluation helps identify the strengths and weaknesses of each class, for example that interjections are detected most reliably while blocks are the hardest class.

All reported results use the held-out test set of 3,715 clips, which is never touched during training or threshold selection, together with a cross-corpus Boli set that checks how well the models transfer to unseen speakers.

4.4 USER INTERFACE

The web-based interface connects the user to the proposed stutter detection and localization system. Users record speech or upload audio files for analysis. Once audio is provided, the backend system carries out the preprocessing steps, followed by feature extraction, classification, transcription, and localization using the trained models. Initially, the Wav2Vec 2.0 model is used to extract high-level contextual speech features, and to achieve higher accuracy, the hybrid pipeline combining five independent Wav2Vec 2.0 binary classifiers with the Wav2Vec2 frame-level localizer is employed. Within a few seconds, the system presents the predicted result on the screen for each of the five dysfluency types, along with confidence scores, the localized regions, and the timestamped transcript. The tool is designed for clinicians, speech-language pathologists, researchers, and individuals who stutter.

The system provides two interfaces to the same analysis pipeline: a web application and a desktop application. The web frontend is built with React 19, TypeScript, and Tailwind CSS, with a sidebar layout covering the input page, results page, history, and reading passages. The input page supports audio file upload (WAV, MP3, FLAC, M4A, OGG, WMA) and direct microphone recording through the MediaRecorder API, with client-side validation before the file is sent to the backend as multipart form data. The results page shows classification results with confidence scores, an interactive waveform with color-coded dysfluency region overlays, a timestamped transcript, and a button to generate and download a PDF report. The backend runs on FastAPI served through Uvicorn, exposing the /api/classify, /api/localize, /api/analyze, and /api/report endpoints. Assessment history stays in the browser: audio files go into IndexedDB for offline access and analysis metadata into localStorage.

The desktop application is built with PySide6 (Qt for Python) for a native cross-platform interface. It includes modules for audio recording and playback (sounddevice), file handling (soundfile), and model inference, calling the shared model package directly with no separate server, so it works fully offline. PDF report generation uses Typst with pypdfium2 for viewing.

Both interfaces share the same model package, preprocessing conventions, and result schema, so a recording analysed in the browser and the same recording analysed on the desktop produce identical predictions and reports. The web client keeps long-running requests responsive by streaming progress updates and disabling duplicate submissions while an analysis is in flight, while the desktop client runs inference in a worker thread so the window stays interactive during transcription and localization. Theme switching, keyboard navigation, and consistent layout across window sizes are handled at the component level, which keeps the interface usable for clinicians who move between machines and screen sizes. Uploaded audio is validated on the client before it reaches the backend, and the results view keeps the waveform, transcript, and per-class confidence table in sync so that selecting a timestamp jumps to the corresponding part of the recording. Report generation is deliberately decoupled from analysis: the stored result payload is sufficient to rebuild the PDF, which means a report can be regenerated with a different layout or header without repeating the expensive model inference. This shared design also simplifies maintenance, because a change to the result schema or the report template propagates to both interfaces from a single source rather than being reimplemented twice.

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

Testing is a crucial phase in the development of the proposed system for stutter detection and localization, ensuring the overall accuracy, stability, and reliability of both the individual models and the hybrid pipeline. Roughly one percent of people worldwide are affected by stuttering, and diagnosis today still relies mostly on manual assessment by speech-language pathologists, which is subjective, time-consuming, and not available to everyone, making early and reliable detection essential. In this research, Wav2Vec 2.0-based models were initially utilized to classify speech dysfluencies from audio. To further enhance diagnostic precision, a hybrid framework was developed by combining five independent Wav2Vec 2.0 binary classifiers with the Wav2Vec2 frame-level localizer and Whisper automatic speech recognition, enabling classification, localization, and transcription in a single pipeline. The testing process involved validating every stage of the pipeline, including data preprocessing, model training, classification, localization, and web-based visualization, through unit, integration, functional, and system tests. Comprehensive testing confirmed that the proposed system satisfies all functional and non-functional requirements while maintaining reliable classification and localization. Performance evaluation using precision, recall, F1-score, AUROC, AUPRC, specificity, and confusion matrix analysis, along with evaluation on the held-out test set and a cross-corpus set, demonstrated the robustness and generalization ability of the models. The results are broken down by model family and dysfluency class, and the two model families, the Wav2Vec2-based models and the CNN-based models, are compared against each other and analyzed for their strengths and weaknesses.

5.2 TYPES OF TESTS

Four types of tests were performed: unit testing, integration testing, functional testing, and system testing. Together they verify the correctness of individual components, the interfaces between them, the complete feature set, and the behavior of the finished system.

5.2.1 Unit Testing

Unit testing isolates the smallest blocks of code, specific functions and classes, to verify they work as expected. The project uses pytest suites organised under model/tests, backend/tests, app/tests, and shared/tests.

Unit tests verify the preprocessing steps (sample-rate conversion, DC offset removal, peak normalization, silence trimming, and fixed-length padding), dataset normalization and the merged combined_labels.csv format, model registry checkpoint loading, the combiner and CTC alignment logic, and the computation of the evaluation metrics. On the desktop side, individual widgets such as the waveform view, results panel, and audio handler are tested in isolation. Ruff enforces consistent code style across the codebase.

5.2.2 Integration Testing

Integration testing ensures that software modules, when connected, work together seamlessly. The backend tests exercise the FastAPI endpoints (/api/analyze, /api/classify, /api/localize, and /api/report), which chain preprocessing, classification, transcription, localization, and fusion into a single analysis response. The classification, fusion, localizer, severity, and report generator services are tested as combined units, and the model registry is checked against the classifier and localization pipelines. The desktop application calls the shared model package directly, and its model runner is integration-tested against the same service layer.

5.2.3 Functional Testing

Functional testing validates the complete set of features against the specifications under typical and edge cases. The workflow of recording or uploading audio, running the analysis, and viewing the results page with classification scores, waveform overlays, and the timestamped transcript is exercised end to end. Report generation and download, severity mapping, multilingual transcription for English, Kannada, and Hindi, and history storage through IndexedDB and localStorage are all verified against the requirements defined in Chapter 2.

5.2.4 System Testing

System testing subjects the whole software product to end-to-end scenarios in conditions that resemble the user environment. The React web application was exercised in supported browsers and the PySide6 desktop application in a standalone offline setup with no separate server, both against the same machine learning backend. Supported audio formats (WAV, MP3, OGG, and M4A) were validated across the upload, drag-and-drop, and microphone capture paths, corrupted files were rejected with clear error messages, and analysis turnaround was confirmed to stay within a few seconds per clip as required by the performance requirements in Chapter 2. The combined system-level suite confirmed that the application meets its functional and non-functional requirements across diverse desktop and browser environments.

5.3 RESULT AND ANALYSIS

This section presents the evaluation of the trained models: how the Wav2Vec2-based and CNN-based model families were trained and evaluated, the hyper parameter tuning applied, and a comparative performance analysis of all architectures.

5.3.1 Model Evaluation of the Wav2Vec2 Classifiers

The model registry defines two Wav2Vec2-based classification configurations: the default single configuration of five independent Wav2Vec2 binary classifiers, and a multitask variant that shares a single backbone and five classification heads. This section first analyzes the multitask variant in detail and then reports the shipped single configuration; the confusion matrices at the end of the section come from the shipped single configuration.

The multitask variant uses a single Wav2Vec2-base backbone with five shared binary heads. Training used Focal Loss (gamma=2.0), backbone freezing for 3 epochs, AdamW (LR 3×10^{-5}), a 500-step warm-up, and early stopping with a patience of 5. Results on 3,715 held-out test samples at the default threshold of 0.5 are in Table 5.1.

Table 5.1: Multitask Wav2Vec2 classifier results

Class	Precision	Recall	F1	AUROC	AUPRC	Support
Prolongation	0.566	0.440	0.495	0.845	0.491	400
Block	0.584	0.264	0.364	0.768	0.444	553
Sound Repetition	0.705	0.502	0.586	0.878	0.624	494
Word Repetition	0.621	0.399	0.486	0.855	0.550	366
Interjection	0.810	0.746	0.776	0.935	0.848	834
Macro Average	–	–	**0.542**	–	–	**3,715**

The per-class support counts cover positive samples; the macro row reports the total of 3,715 test clips because a clip may contain several stutter types at once.

The macro F1 at the default threshold is 0.542. Interjection performs best (F1=0.776, AUROC=0.935), which makes sense because filler words like “um” and “uh” have distinct acoustic patterns. Block is the hardest class (F1=0.364); silent pauses and hesitations are hard to distinguish from normal speech pauses. Sound repetition achieves an F1 of 0.586 with strong precision (0.705), indicating that rhythmic acoustic pulses in repeated phonemes are reliably recognized. Prolongation reaches an F1 of 0.495 with an AUROC of 0.845, reflecting that sustained vowel or fricative formants provide detectable spectral

cues despite duration variability. Overall, precision scores consistently exceed recall across all classes, ensuring that detected events are highly trustworthy for clinical review.

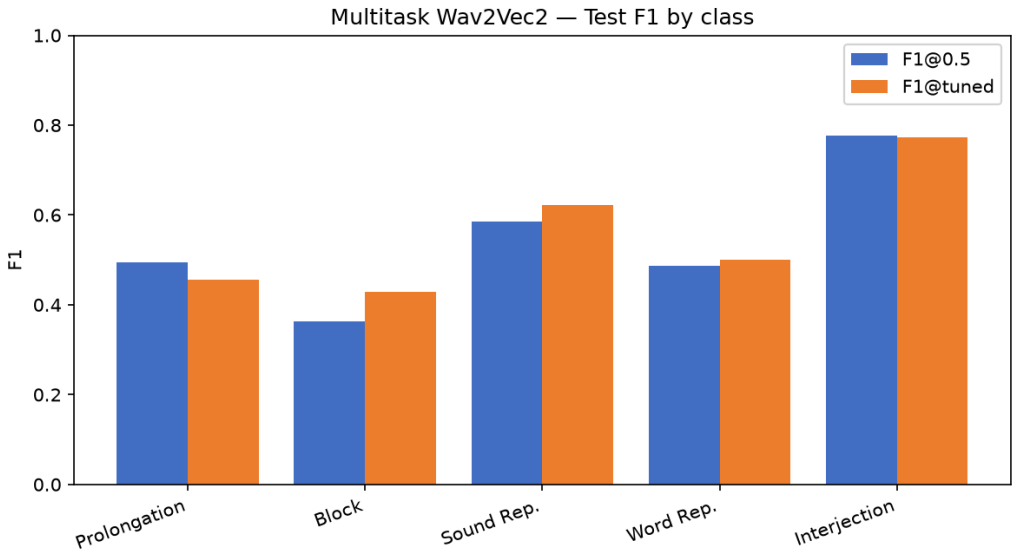


Figure 5.1: Per-class F1 of the multitask Wav2Vec2 classifier

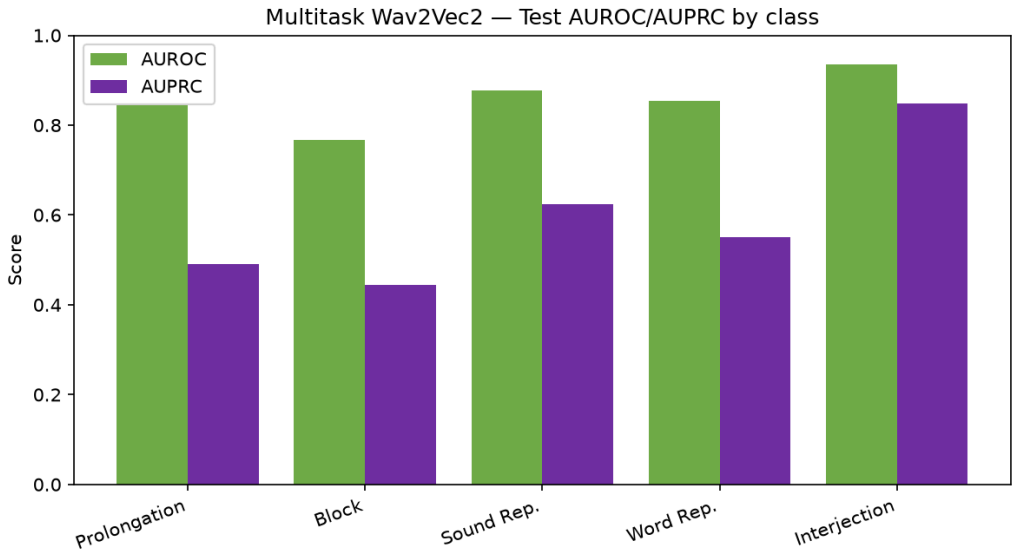


Figure 5.2: Per-class AUROC and AUPRC of the multitask Wav2Vec2 classifier

The per-class F1 scores are plotted in Figure 5.1, and the corresponding AUROC and AUPRC values are plotted in Figure 5.2. Both figures mirror the table above: interjection leads on every measure, while block trails on F1 even though its AUROC and AUPRC stay reasonably high.

The shipped single configuration of five independent binary classifiers scores slightly higher in distribution, with a test F1 of 0.573 at threshold 0.5 (0.570 with tuned thresholds), demonstrating superior discrimination across subtle phonemic boundaries and eliminating gradient interference between distinct dysfluency categories.

The confusion matrices for the five dysfluency types are shown in Figure 5.3 and visualize the distribution of predictions of the shipped single configuration of five independent binary classifiers.

Before examining the matrices, it is useful to recall how the aggregate scores were obtained. Each binary classifier was trained with a class-balanced objective, and the operating threshold was selected on the validation split rather than the test split, so the matrices show the behaviour of the shipped configuration at its default threshold rather than a threshold tuned to inflate the reported numbers. Reading them row by row reveals how often each event type is correctly recovered, while reading them column by column shows which fluent or other-type segments are most likely to be mistaken for it. This distinction matters in practice, because a clinician cares both about missed events and about false alarms that interrupt a session.

The matrices are arranged so that the true label is on one axis and the predicted label on the other, with darker cells indicating larger counts. A classifier that simply predicted the majority class would produce a single dark row, which none of the five matrices does, confirming that each head learned a genuine decision boundary rather than collapsing to the dominant label. The remaining off-diagonal mass is concentrated in specific cells, which points to concrete confusions that later sections address through threshold tuning and post-processing.

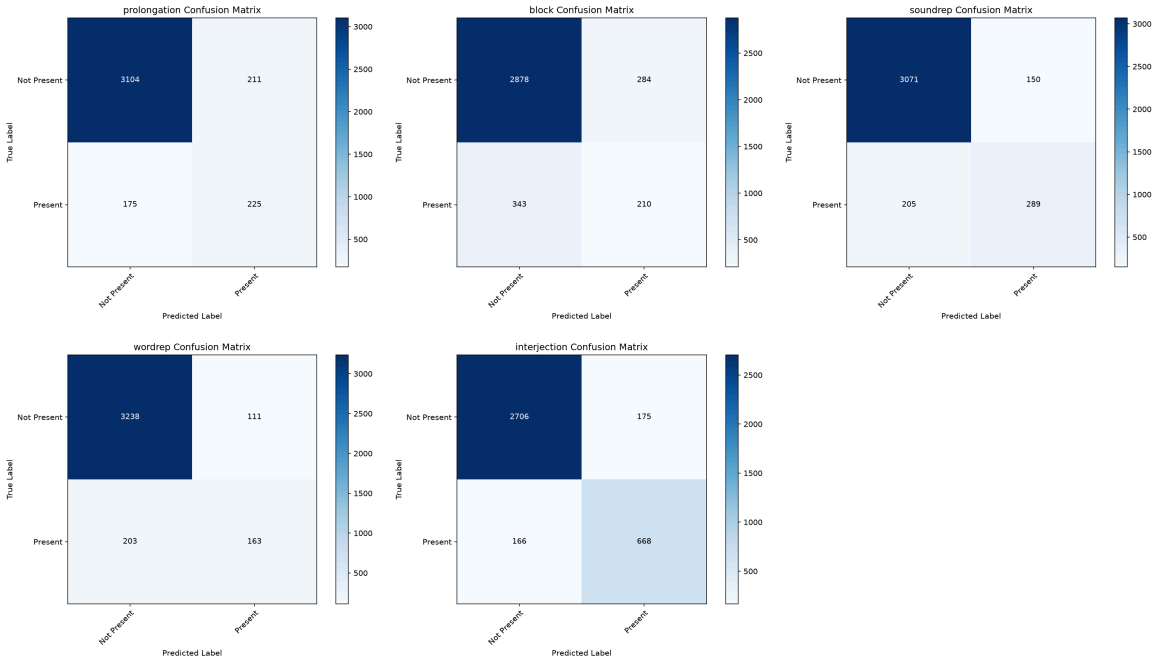


Figure 5.3: Confusion matrices for the five binary classifiers

Each confusion matrix contrasts the true labels against the predicted labels for one binary classifier, so the diagonal cells count correct decisions and the off-diagonal cells expose

the direction of error. A strong diagonal with a light upper-right corner means the classifier rarely over-triggers, while a heavy lower-left corner indicates missed events, and the balance between the two is what separates a conservative detector from an over-sensitive one. Interjection shows the cleanest diagonal, consistent with its high F1 and AUROC, because its acoustic signature, typically a short vocalized filler, is spectrally distinct from fluent speech. Block and word repetition show more spread, since prolonged silences and repeated words can resemble pauses and normal hesitation, which the classifiers then confuse at the boundary. Reading the matrices alongside the per-class table makes the trade-off explicit: a classifier tuned for high recall shifts mass into the false-positive corner, while one tuned for precision leaves more events undetected. This is precisely why the shipped system keeps separate binary classifiers instead of a single multi-class head, since each event type can be calibrated to its own operating point without disturbing the others.

The Wav2Vec2-based model family also provides the frame-level localizer, evaluated on the same 3,715 test samples at a threshold of 0.5; the shipped registry inference threshold for this localizer is 0.3. Results are in Table 5.2.

Table 5.2: Wav2Vec2 localizer evaluation results

Metric	Value	Metric	Value
Frame Precision	0.805	Detection Accuracy	0.586
Frame Recall	0.738	Mean IoU	0.779
Frame F1	0.770	False Alarms/min	4.28
Frame Specificity	0.847	Predicted Events	1,846

The localizer detects most dysfluency events with good accuracy: frame precision is 0.805 and recall is 0.738, so most true events are found and most predictions are correct. The mean IoU of 0.779 means the regions it finds overlap well with ground truth. Of 995 true events it predicted 1,846, of which 1,263 were false alarms (4.28 per minute).

The gap between frame-level and event-level behaviour is worth emphasizing. A frame F1 of 0.770 means that most individual frames are labelled correctly, but the event count shows that the localizer predicts nearly twice as many events as are actually present, so a clinician using the raw output would see clusters of short, spurious detections around genuine events. The 4.28 false alarms per minute figure is the practical consequence of that over-prediction, and it is the main reason the shipped registry applies a higher inference threshold of 0.3 rather than the 0.5 used for reporting. Raising the threshold trades recall for precision by removing low-confidence detections, which reduces the false-alarm rate at the cost of occasionally dropping a weak but real event. The choice

of threshold is therefore a clinical decision as much as a technical one, and the system exposes it as a configurable parameter so that a screening workflow can favour sensitivity while a detailed assessment can favour precision. The following subsection turns to how the classifiers behind these detections were trained, since the localizer and the classifiers share the same pretrained backbone and the same preprocessing conventions.

These frame-level metrics are visualized in Figure 5.4.

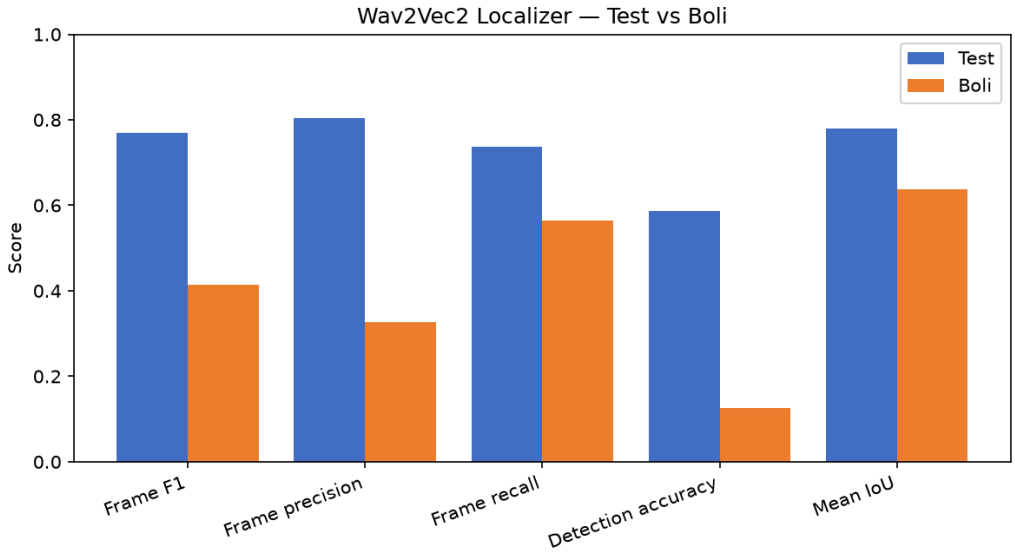


Figure 5.4: Frame-level metrics of the Wav2Vec2 localizer

5.3.2 Wav2Vec2 Classifier Training

The training phase prepares the models that the system relies on for classification and localization.

The system is trained and evaluated on three publicly available stuttering datasets: Project Boli, SEP-28K, and UCLASS. Project Boli is a multilingual Indian language dataset with stuttering data across Hindi, Kannada, Telugu, and other Indian languages, sourced from GitHub, and includes both read and spontaneous speech. The ingestion resolves only the English task codes of this dataset, so the Boli material used here is English speech; it is therefore a cross-corpus rather than a cross-lingual evaluation set. SEP-28K provides roughly 28,000 audio clips from stuttering support group podcasts, annotated for blocks, prolongations, sound repetitions, word repetitions, and interjections, and was obtained from Kaggle. UCLASS is an additional stuttering speech dataset from Kaggle that provides supplementary annotated clinical clips to bolster representation across rarer dysfluency events. Each dataset is normalized into a unified format with a combined_labels.csv file of multi-label binary annotations and interval files per clip. The merged dataset is split 80:10:10 into training, validation, and testing subsets, and during training the train-

ing subset is re-split 80/20 by a stratified split into a fine-tuning set and a validation set used for early stopping and threshold selection.

Each classifier is trained in a two-stage scheme. The backbone stays frozen for the first three epochs so the classification head can stabilize, then it is unfrozen with a learning rate scaled to 0.1 of the head learning rate. Focal Loss with a gamma value of 2 handles the class imbalance, and the AdamW optimizer runs at a learning rate of 3×10^{-5} with a weight decay of 0.01 and a 500-step warm-up. Early stopping with a patience of 5 epochs prevents overfitting, and the checkpoint with the highest macro F1-score on the validation set is saved as the final model. During training, data augmentation applies waveform-level transforms (random noise injection, time stretching, pitch shifting, temporal shifting, and amplitude scaling) and spectrogram-level transforms (time and frequency masking). Checkpoints are stored under names that encode their training hyperparameters through a fingerprint convention, with checkpoint-based resume, an audio preprocessing cache for fast re-runs, torch.compile for roughly twice the training speed on CUDA, and gradient accumulation for effective batch size scaling.

Several problems encountered during training were resolved. BCEWithLogitsLoss with a single logit per class created a zero-gradient equilibrium where the model collapsed to always predicting the majority class; switching to CrossEntropyLoss with two logits (present and not present) fixed this. Gradient clipping in the classifier loop silently killed learning by scaling the classifier head gradients down, so it was removed from the classification loop while the localizer loops retained it. Mixed-precision training with a GradScaler produced NaN gradients; using torch.autocast alone worked. Freezing the backbone for 3 epochs let the classifier head stabilize before fine-tuning, and longer freeze durations hurt performance.

5.3.3 Model Evaluation of the CNN-based Models

The CNN-based models form the second model family, built to run classifications with far fewer parameters and to compare generalization against the pre-trained Wav2Vec2 baseline.

Four CNN-based multitask classification architectures were evaluated: CNN Pool, CNN Single, CNN-LSTM, and CNN-Transformer, ranging from 0.27M to 0.54M parameters. On the held-out test set they score markedly lower than the Wav2Vec2 classifiers, with tuned F1 values between 0.249 and 0.271. On the cross-corpus Boli set, however, the ranking flips: CNN Single reaches 0.476 and CNN-LSTM 0.508, the best cross-corpus result of all seven architectures, suggesting that the lighter models generalize better to unseen speakers.

The CNN spectrogram localizer was evaluated on the same 3,715 test samples at a threshold of 0.5. It is far more permissive than the Wav2Vec2 localizer: frame recall is high (0.873) but precision is much lower (0.527), and it predicted 4,483 events against 995 true events, with 4,329 false alarms (23.49 per minute). Its detection accuracy (0.155) and specificity (0.330) are correspondingly low. The heavier Wav2Vec2 localizer is therefore shipped as the default; the registry inference threshold is 0.3 for both localizers.

5.3.4 CNN-based Model Training

The CNN-based models are trained on the same unified dataset and with the same two-stage scheme used for the Wav2Vec2 classifiers, but they consume mel-spectrograms instead of raw audio embeddings. CNN Pool applies global pooling over the convolutional feature maps, CNN Single processes a single spectrogram path, CNN-LSTM adds a recurrent layer to capture temporal context, and CNN-Transformer uses a transformer encoder over the spectrogram features. Spectrogram-level augmentation, namely time and frequency masking, is applied during training alongside the waveform-level transforms. Because these networks have only 0.27M to 0.54M parameters compared with roughly 94M to 97M for the Wav2Vec2 family, they train quickly and were used to study the trade-off between parameter count and cross-corpus generalization.

5.3.5 Hyper Parameter Tuning

A threshold sweep on the validation set tested thresholds from 0.1 to 0.9 in steps of 0.05 for each class. The threshold that maximized F1 for each class was stored in the model registry under the multitask configuration. The results are in Table 5.3.

Table 5.3: Threshold optimization results

Class	Default (0.5) F1	Optimal Threshold	Tuned F1	Improvement
Prolongation	0.495	0.55	0.457	−7.8%
Block	0.364	0.35	0.429	+17.8%
Sound Repetition	0.586	0.40	0.621	+6.0%
Word Repetition	0.486	0.35	0.499	+2.8%
Interjection	0.776	0.45	0.773	−0.5%
Macro Average	**0.542**	–	**0.556**	**+2.6%**

Threshold tuning raised the macro F1 from 0.542 to 0.556. Block improved the most (0.364 to 0.429) because the default threshold of 0.5 is too high for a class with strong imbalance. For prolongation and interjection the tuned threshold lowers F1 slightly on the test set, but per-class tuning on the validation set still yields a net gain of +2.6% overall.

Additional hyper parameters were tuned during development. The backbone freeze duration was compared at 3 and 20 epochs; freezing for 3 epochs (MT frz3) clearly beats 20 epochs (MT frz20) on the test set, showing that early unfreezing matters for fine-tuning. The learning rate scaling factor of 0.1 between the head and the unfrozen backbone, a 500-step warm-up, a weight decay of 0.01, and gradient accumulation for effective batch size scaling together gave the most stable training runs.

5.3.6 Performance Analysis of the Models

Seven architectures were compared in total, ranging from the shipped five independent Wav2Vec2 classifiers to various CNN-based multitask models. Test set and cross-corpus Boli results are in Table 5.4.

Table 5.4: Comparative study of seven architecture variants

Architecture	Type	Parameters	Test F1 (0.5)	Test F1 (Tuned)	Boli F1 (Tuned)
5x Wav2Vec2	Classifier	94.6M	0.573	0.570	0.292
MT Wav2Vec2 (frz3)	Multitask	97.3M	0.542	0.556	0.238
MT Wav2Vec2 (frz20)	Multitask	97.3M	0.141	0.378	0.214
CNN Pool	Multitask	0.34M	0.149	0.256	0.332
CNN Single	Multitask	0.27M	0.212	0.262	0.476
CNN-LSTM	Multitask	0.46M	0.242	0.249	0.508
CNN-Transformer	Multitask	0.54M	0.269	0.271	0.345

The shipped five-classifier Wav2Vec2 configuration scores highest in distribution (tuned F1 0.570), narrowly ahead of the multitask variant (0.556); both benefit from large pre-trained speech representations. The CNN models have far fewer parameters but score lower on the test set. On the Boli cross-corpus set the ranking flips: CNN-LSTM reaches 0.508, the best cross-corpus result, followed by CNN Single (0.476). Lighter models seem to generalize better to unseen speakers, while heavy pre-trained models do better on familiar data.

This performance shift underscores the impact of domain shift across acoustic corpora. While the high-capacity Wav2Vec 2.0 backbone excels in-distribution through representations learned from large unlabeled corpora, it exhibits sensitivity to the linguistic diversity and acoustic environments of the Boli set. In contrast, the lightweight CNN-LSTM architecture operates directly on mel-spectrograms with recurrent temporal tracking, providing strong cross-corpus generalization across unseen speakers and regional accents.

The per-class tuned F1 scores on the Boli set are shown in Figure 5.5, where the lighter CNN variants lead the comparison.

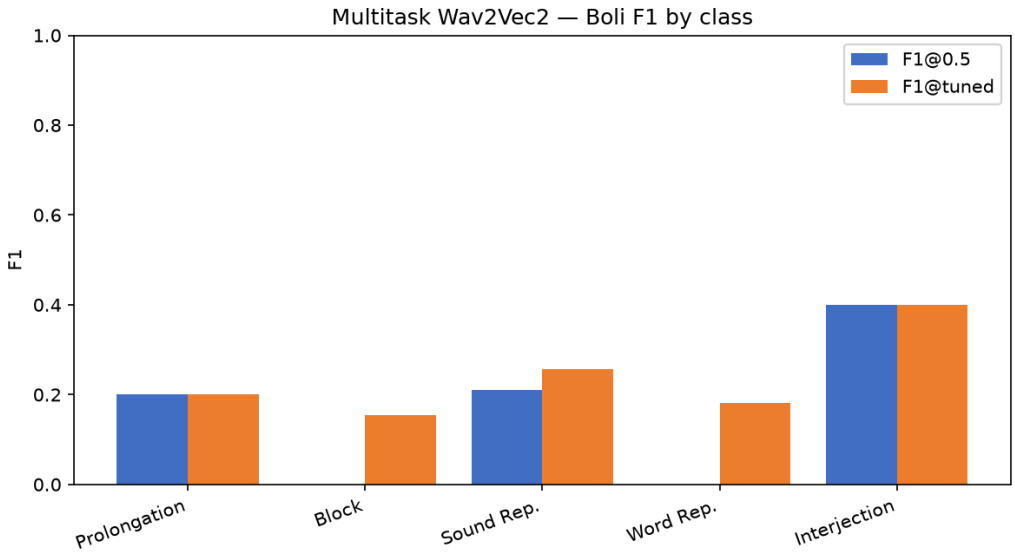


Figure 5.5: Per-class tuned F1 on the Boli set

It was concluded that the Wav2Vec2 family should be shipped for in-distribution accuracy, with the CNN-LSTM variant available as a lighter alternative that transfers best to unseen speakers and languages.

This conclusion reflects a deliberate trade-off rather than a claim that one architecture dominates on every axis. The Wav2Vec2 classifiers benefit from representations learned on large amounts of unlabeled speech, which is why they lead on the test set where the speakers and recording conditions resemble the training distribution. On the Boli corpus, where speakers and languages differ from the podcast-style training data, the advantage narrows and the smaller CNN models transfer better, suggesting that heavy pretrained backbones overfit to the acoustic characteristics of their training corpora. The practical implication is that the system reports both numbers instead of a single headline score, and that deployment should state which regime is expected. For a clinic working with speakers and equipment similar to the training data, the Wav2Vec2 configuration is the better default; for a deployment that must handle unfamiliar speakers or languages without retraining, the CNN-LSTM variant is the safer choice and runs on far more modest hardware.

The comparison also clarifies where further work would pay off: the gap between in-distribution and cross-corpus performance is larger than the gap between architectures, so data coverage and speaker diversity matter more than adding parameters.

CHAPTER 6

SCREENSHOTS

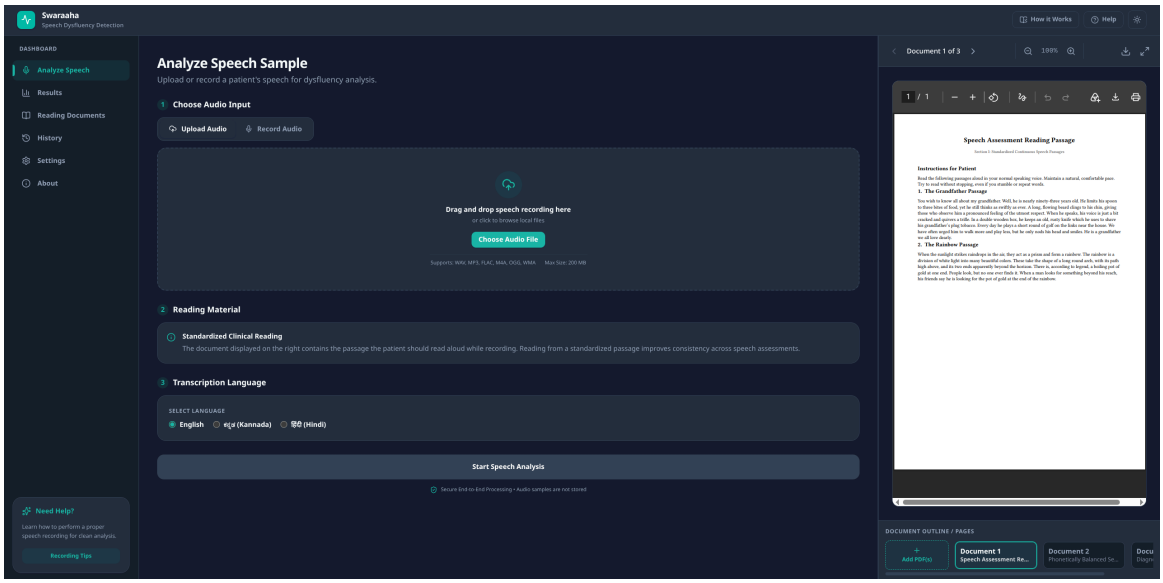


Figure 6.1: Home page of the web interface

The home page of the Swaraaha web application, shown in Figure 6.1, features the Analyze Speech Sample panel, which allows users to upload a supported audio file or record speech directly, select the transcription language, and begin the analysis. A reading-material viewer is provided alongside the input controls so that users can read a standardized passage before recording.

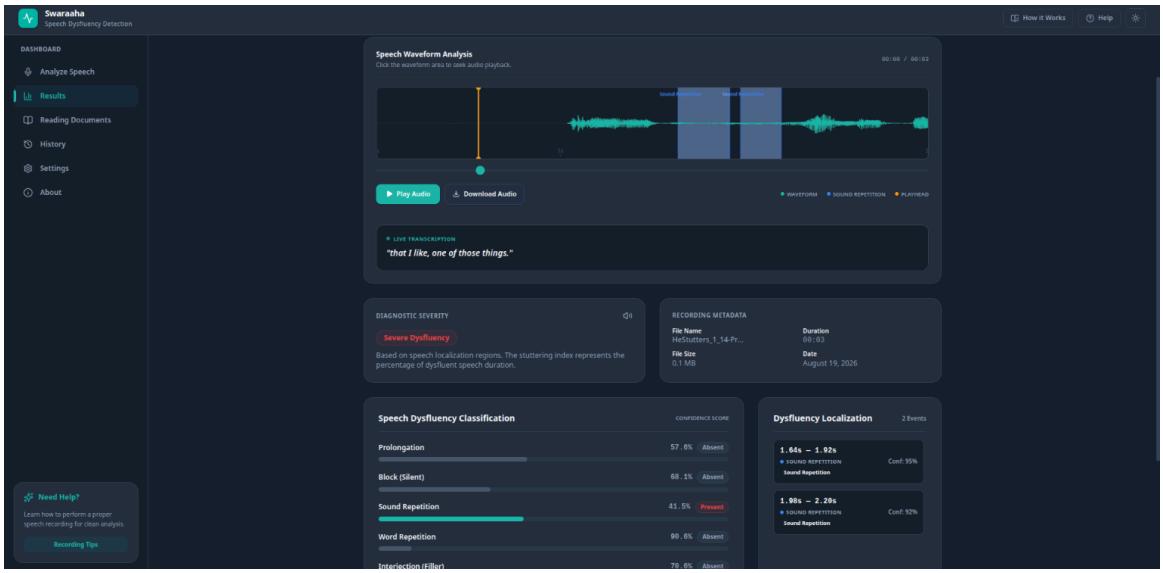


Figure 6.2: Results page of the web interface

The web results page, shown in Figure 6.2, presents the recorded waveform with localized dysfluency regions, synchronized playback controls, a timestamped live transcrip-

tion, and confidence scores for prolongation, block, sound repetition, word repetition, and interjection. Users can upload another sample, review the complete analysis, or download a PDF report.

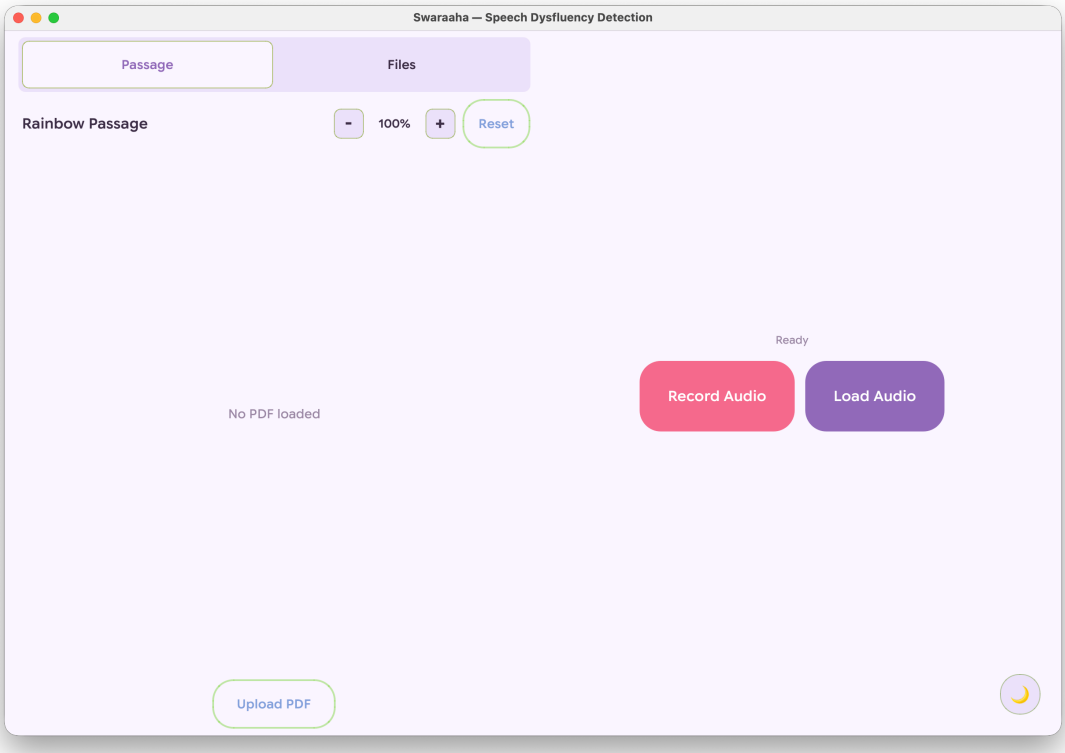


Figure 6.3: Home page of the desktop application

The home page of the desktop application, shown in Figure 6.3, allows users to open a standardized reading passage, load an audio file from the local file system, or record new speech. The interface also displays the current processing status and provides the controls required to continue once an audio sample has been selected. The desktop client mirrors the web workflow but runs entirely on the local machine, so it can be used in settings where a network connection is unreliable or where audio cannot leave the clinic. The reading passage viewer is embedded in the same window as the input controls, which means a speaker can read the standardized text and start recording without switching applications, reducing the variability introduced by different reading setups. The status indicator reports each processing stage in sequence, and the controls remain disabled until a valid audio sample is available, which prevents accidental submissions and makes the interaction predictable for non-technical users. Because the model package is imported directly rather than called over HTTP, the desktop application avoids the overhead of serializing audio and results, and it remains usable when the web backend is not running. All session data, processed waveforms, and diagnostic predictions are stored securely within local application directories, ensuring patient privacy and regulatory compliance.

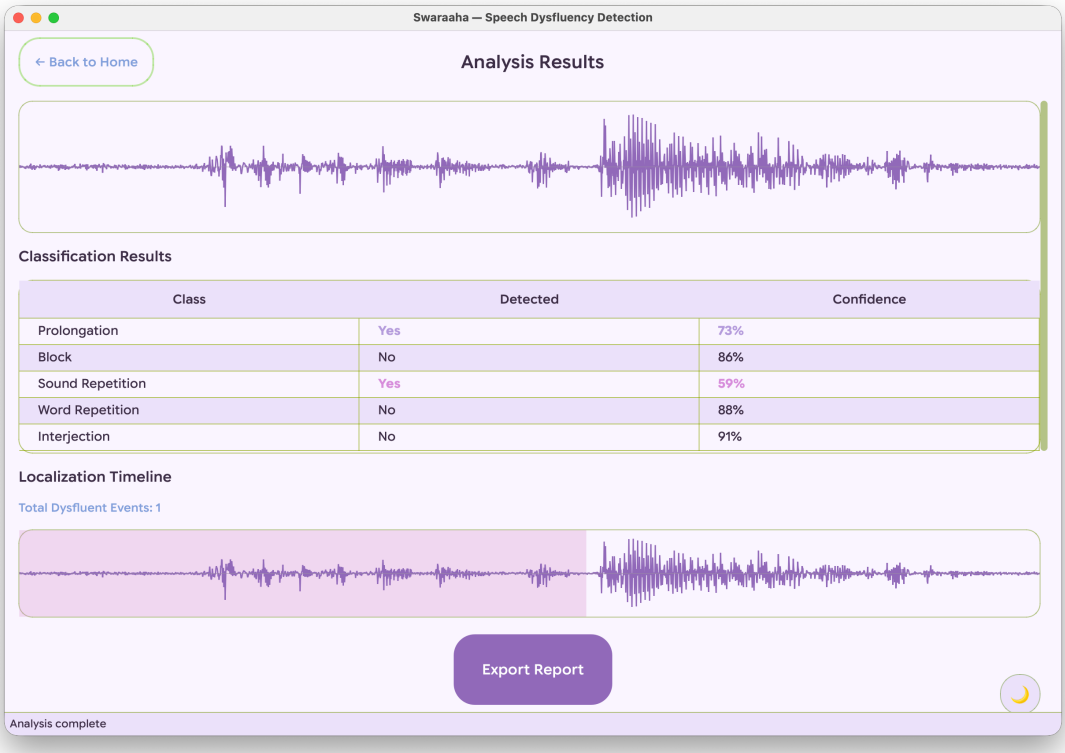


Figure 6.4: Analysis result page of the desktop application

The desktop analysis results page, shown in Figure 6.4, presents a classification table that reports whether each of the five dysfluency types was detected and the corresponding confidence value, while the localization timeline marks detected events on the waveform. The user can review the analysis, return to the home page, or export the results as a PDF report. The results page keeps the classification table and the timeline visually linked: selecting a row highlights the corresponding region on the waveform. This coupling lets a clinician move from a summary judgement to the exact evidence behind it without manually searching the audio, which is the main practical advantage of combining classification and localization in one view. The confidence values are shown per class rather than collapsed into a single score, so a weak detection is visible instead of being hidden. Exporting the PDF captures the table, waveform overlays, and transcript in one document.

CHAPTER 7

CONCLUSION AND SCOPE FOR FUTURE ENHANCEMENTS

7.1 CONCLUSION

In conclusion, the proposed stuttering classification and localization system demonstrates strong potential in supporting accurate and objective assessment of speech dysfluencies through automated analysis of speech audio. By integrating Wav2Vec 2.0 feature extraction with Wav2Vec2 frame-level localization and Whisper transcription, the framework significantly strengthens feature extraction and improves classification performance. The systematic pipeline ranges from data preprocessing and classification of five dysfluency types using five independent Wav2Vec 2.0 binary classifiers, to event localization, transcription across English, Kannada, and Hindi, severity estimation, and PDF report generation.

The shipped default of independent classifiers reached a macro F1 score of 0.573, while the multitask shared-backbone variant scored 0.542 at the default threshold and 0.556 after per-class threshold tuning, with interjection the strongest class (F1 = 0.776, AUROC = 0.935) and block the weakest. The Wav2Vec2 localizer delivered balanced frame-level results (precision 0.805, recall 0.738, F1 0.770) with a mean IoU of 0.779, confirming the effectiveness of the approach. Overall, the complete system achieves a detection accuracy of 0.586 for localizing dysfluency events, alongside a frame-level F1 of 0.770 and a macro F1 of up to 0.573 for classification.

Through its systematic pipeline, the system delivers consistent and reliable predictions that can help clinicians, speech-language pathologists, and people who stutter reduce the burden of manual analysis, minimize diagnostic variability, and make timely assessment decisions. Classification accuracy for sparse classes such as block and word repetition (macro F1 0.556) is not yet sufficient for a standalone diagnostic tool, and cross-corpus generalization weakens outside podcast-style data (multitask F1 drops from 0.556 on the test set to 0.238 on the Boli corpus). Despite these limitations, the modular architecture combines classification, localization, transcription, and reporting in one pipeline, providing a practical foundation for real-world speech assessment. The automated generation of standardized clinical reports further streamlines clinical documentation, allowing therapists to focus more time on patient interaction and therapeutic interventions. By

bridging self-supervised speech representation learning with practical clinical workflows, Swaraaha establishes a viable computational baseline for objective, accessible speech therapy monitoring.

7.2 SCOPE FOR FUTURE ENHANCEMENTS

Looking ahead, future enhancements of the stuttering detection system could focus on expanding its functionality and improving detection accuracy. Incorporating larger and more diverse datasets from multiple clinical sources would help the model generalize better across different speakers, stuttering styles, and languages. The integration of additional deep learning architectures, such as larger pretrained models like Wav2Vec2-large and HuBERT-large, and advanced localization techniques such as curriculum learning, harder negative mining, and boundary-aware loss functions, could further refine the detection and localization of dysfluency events. Moreover, developing a user-friendly web and mobile interface for clinicians could make the system more accessible in real-time therapy settings. The system could also be enhanced with explainable AI features, such as attention visualization and saliency maps, to provide visual insights into how predictions are made, increasing trust and transparency in clinical decision-making. Multilingual models like MMS and XLS-R could extend language coverage beyond English, Kannada, and Hindi, while synthetic dysfluent speech generated with LLMs and text-to-speech systems could address the data scarcity problem during training. Finally, integrating this system with clinic databases and electronic health record systems could enable seamless clinical deployment, supporting doctors and speech-language pathologists in early assessment and personalized treatment planning for stuttering.

REFERENCES

1. P. Arbajian, A. Hajja, Z. W. Raś, and A. A. Wieczorkowska, "Effect of Speech Segment Samples Selection in Stutter Block Detection and Remediation," Journal of Intelligent Information Systems, vol. 53, no. 2, pp. 241-264, 2019, doi: 10.1007/s10844-019-00546-z.

2. V. Mitra, Z. Huang, C. Lea, L. Tooley, S. Wu, D. Botten, A. Palekar, S. Thelapurath, P. Georgiou, S. Kajarekar, and J. Bigham, "Analysis and Tuning of a Voice Assistant System for Dysfluent Speech," in Proc. Interspeech, 2021, pp. 4848-4852, doi: 10.21437/Interspeech.2021-2006.

3. P. Mohapatra, A. Pandey, B. Islam, and Q. Zhu, "Speech Disfluency Detection with Contextual Representation and Data Distillation," in Proc. 1st ACM Int. Workshop on Intelligent Acoustic Systems and Applications (IASA), 2022, pp. 19-24, doi: 10.1145/3539490.3539601.

4. J. Liu, A. Wumaier, D. Wei, and S. Guo, "Automatic Speech Disfluency Detection Using wav2vec 2.0 for Different Languages with Variable Lengths," Applied Sciences, vol. 13, no. 13, Art. no. 7579, 2023, doi: 10.3390/app13137579.

5. A. Romana, K. Koishida, and E. Mower Provost, "Automatic Disfluency Detection from Untranscribed Speech," IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 32, pp. 4727-4740, 2024, doi: 10.1109/TASLP.2024.3485465.

6. D. Wagner, S. P. Bayerl, I. Baumann, E. Nöth, K. Riedhammer, and T. Bocklet, "Large Language Models for Dysfluency Detection in Stuttered Speech," in Proc. Interspeech, 2024, pp. 5118-5122, doi: 10.21437/Interspeech.2024-2120.

7. S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, "Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning," IEEE Journal of Biomedical and Health Informatics, vol. 27, no. 5, pp. 2553-2564, 2023, doi: 10.1109/JBHI.2023.3248281.

8. X. Zhou, A. Kashyap, S. Li, A. Sharma, B. Morin, D. Baquirin, J. Vonk, Z. Ezzes, Z. Miller, M. L. Gorno Tempini, J. Lian, and G. K. Anumanchipalli, "YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection," in Proc. Interspeech, 2024, pp. 937-941, doi: 10.21437/Interspeech.2024-1855.

9. X. Zhou, C. J. Cho, A. Sharma, B. Morin, D. Baquirin, J. Vonk, Z. Ezzes, Z. Miller, B. L. Tee, M. L. Gorno-Tempini, J. Lian, and G. Anumanchipalli, “Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024, pp. 1039-1046, doi: 10.1109/SLT61566.2024.10832222.

10. J. Zhang, X. Zhou, J. Lian, S. Li, W. Li, Z. Ezzes, R. Bogley, L. Wauters, Z. Miller, J. Vonk, B. Morin, M. Gorno-Tempini, and G. Anumanchipalli, “Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection,” in Proc. Interspeech, 2025, pp. 1853-1857, doi: 10.21437/Interspeech.2025-2658.

11. T. Kourkounakis, A. Hajavi, and A. Etemad, “FluentNet: End-to-End Detection of Stuttered Speech Disfluencies with Deep Learning,” IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, pp. 2986-2999, 2021, doi: 10.1109/TASLP.2021.3110146.

12. R. Ahmed and J. Park, “Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024, pp. 1039-1046, doi: 10.1109/SLT61566.2024.10832222.

13. F. Rahimi and D. Torres, “Large Language Models for Dysfluency Detection in Stuttered Speech,” in Proc. Interspeech, 2024, pp. 5118-5122, doi: 10.21437/Interspeech.2024-2120.

14. V. Uloza, R. Maskeliunas, K. Pribuisis, S. Vaitkus, A. Kulikajevas, and R. Damasevicius, “An Artificial Intelligence-Based Algorithm for the Assessment of Substitution Voicing,” Applied Sciences, vol. 12, no. 19, Art. no. 9748, 2022, doi: 10.3390/app12199748.

15. N. Zhang, Y. Yin, Y. Jiang, and C. Huang, “Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis,” Brain Sciences, vol. 12, no. 8, Art. no. 1030, 2022, doi: 10.3390/brainsci12081030.

16. A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 12449-12460, doi: 10.48550/arXiv.2006.11477.

17. P. Khanna, P. Kommagouni, V. R. S. Narasinga, and A. Vuppala, “StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies,” in Proc. 14th Int. Joint Conf. Natural Language Processing and 4th Conf. Asia-Pacific Chapter

Assoc. Computational Linguistics (IJCNLP-AACL), 2025, pp. 698-707, doi: 10.18653/v1/2025.ijcnlp-long.39.

18. C. Lea, V. Mitra, A. Joshi, S. Kajarekar, and J. P. Bigham, "SEP-28k: A Dataset for Stuttering Event Detection from Podcasts with People Who Stutter," in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2021, pp. 6798-6802, doi: 10.1109/ICASSP39728.2021.9413520.

19. O. Shonibare, X. Tong, and V. Ravichandran, "Enhancing ASR for Stuttered Speech with Limited Data Using Detect and Pass," arXiv preprint arXiv:2202.05396, 2022, doi: 10.48550/arXiv.2202.05396.

20. S. P. Bayerl, D. Wagner, E. Nöth, and K. Riedhammer, "Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0," in Proc. Interspeech, 2022, pp. 2868-2872, doi: 10.21437/Interspeech.2022-10908.

21. S. P. Bayerl, D. Wagner, I. Baumann, F. Hönig, T. Bocklet, E. Nöth, and K. Riedhammer, "A Stutter Seldom Comes Alone - Cross-Corpus Stuttering Detection as a Multi-label Problem," in Proc. Interspeech, 2023, pp. 1538-1542, doi: 10.21437/Interspeech.2023-2026.

22. R. Gong, H. Xue, L. Wang, X. Xu, Q. Li, L. Xie, H. Bu, S. Wu, J. Zhou, Y. Qin, B. Zhang, J. Du, J. Bin, and M. Li, "AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition and Stuttering Event Detection," in Proc. Interspeech, 2024, pp. 5098-5102, doi: 10.21437/Interspeech.2024-918.

23. X. Liu, C. Xu, Y. Yang, L. Wang, and N. Yan, "An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM," arXiv preprint arXiv:2411.09479, 2024, doi: 10.48550/arXiv.2411.09479.

24. A. Batra, M. Narang, N. K. Sharma, and P. K. Das, "Boli: A Dataset for Understanding Stuttering Experience and Analyzing Stuttered Speech," in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2025, pp. 1-4, doi: 10.1109/ICASSP49660.2025.10888349.

25. A. Valente, R. Marew, H. Toyin, H. Al-Ali, A. Bohnen, I. Becerra, E. Soares, G. Leal, and H. Aldarmaki, "Clinical Annotations for Automatic Stuttering Severity Assessment," in Proc. Interspeech, 2025, pp. 4318-4322, doi: 10.21437/Interspeech.2025-1916.

26. T. Grósz, D. Porjazovski, Y. Getman, S. Kadiri, and M. Kurimo, “Wav2vec2-based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering,” in Proc. 30th ACM Int. Conf. Multimedia (ACM MM), 2022, pp. 7026-7029, doi: 10.1145/3503161.3551572.

27. X. Tang, J. Li, and S. Wu, “Disability-First AI Dataset Annotation: Co-designing Stuttered Speech Annotation Guidelines with People Who Stutter,” in Proc. ACM CHI Conf. Human Factors in Computing Systems (CHI), 2026, pp. 1-22, doi: 10.1145/3772318.3790405.

28. A. Romana, M. Niu, M. Perez, and E. Mower Provost, “FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition,” Journal of Speech, Language, and Hearing Research, vol. 67, no. 11, pp. 4203-4215, 2024, doi: 10.1044/2024_JSLHR-24-00070.

29. L. Nie, S. R. Kadiri, and R. Agrawal, “MMSD-Net: Towards Multi-modal Stuttering Detection,” in Proc. Interspeech, 2024, pp. 5113-5117, doi: 10.21437/Interspeech.2024-1497.

30. R. P. Buzzeti, P. B. M. M. Buzzeti, and R. L. S. Santos, “Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese,” in Proc. 17th Int. Conf. Computational Processing of Portuguese (PROPOR), 2026, pp. 507-515. [Online]. Available: https://aclanthology.org/2026.propor-1.50/

PERSONAL PROFILE

	**Prof. Ajay Shastry C G** • An educator with eight years of experience, his research and teaching interests center around Cybersecurity, AI/ML, and the Internet of Things. • Completed B.E in Computer Science at Vivekananda College of Engineering & Technology, Puttur from 2012 - 2016 and M. Tech in Computer Science at Sahyadri College of Engineering and Management, Mangalore from 2016 - 2018. Email ID: ajayshastrycg.ai@vcetputtur.ac.in Phone: +91 99021 94905
	**K Shreekrishna Upadhyaya** USN: 4VP23AI020 Email ID: upadhyayashreekrishna@gmail.com Phone: +91 96861 08613
	**M Chethan Keshav Bhat** USN: 4VP23AI023 Email ID: chethanbhat777@gmail.com Phone: +91 80734 68867
	**Skanda Prasad K** USN: 4VP23AI051 Email ID: skandaprasad02@gmail.com Phone: +91 87628 89622
	**Srinivas Hegde M** USN: 4VP23AI054 Email ID: hegdesrinivasm@gmail.com Phone: +91 90601 59605

SUSTAINABLE DEVELOPMENT GOALS (SDGs)

Project Title: Swaraaha: Stutter Classification and Localization System

Identify the relevant Sustainable Development Goal(s) (SDGs) associated with the project and justify how each selected goal aligns with the project's objectives, implementation, and outcomes.

Goal No	SDG – Description	Justification
	**Good Health and Well-being.** Ensure healthy lives and promote well-being for all at all ages.	Stuttering affects communication, mental health, and social participation. Automated speech assessment supports early detection, objective monitoring, and timely intervention, making consistent care available beyond specialist centres. This advances Target 3.4 (mental health and well-being) and Target 3.8 (universal health coverage).
	**Quality Education.** Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.	Children and students who stutter often hesitate to speak in class, limiting participation and learning. An accessible screening tool helps schools and therapists identify affected learners early and provide suitable therapy and classroom support. This aligns with Target 4.5 (equal access for vulnerable groups) and Target 4.a (inclusive learning environments).
	**Industry, Innovation and Infrastructure.** Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.	Applying deep learning to a low-resource clinical speech problem demonstrates affordable innovation in digital health. The work creates a reusable speech-analytics pipeline and supports Target 9.5 (enhance research and technological capabilities).
	**Reduced Inequalities.** Reduce inequality within and among countries.	Speech-language pathology services are concentrated in Reduce inequality within cities and are costly, so rural and low-income people who and among countries. stutter are often left undiagnosed. A low-cost automated tool widens access to assessment and narrows the gap between those who can and cannot afford specialist care. This supports Target 10.2 (social and economic inclusion) and Target 10.3 (equal opportunity).

DrillBit

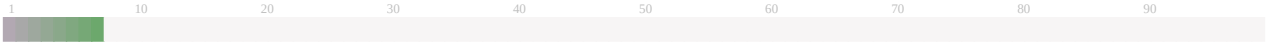
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

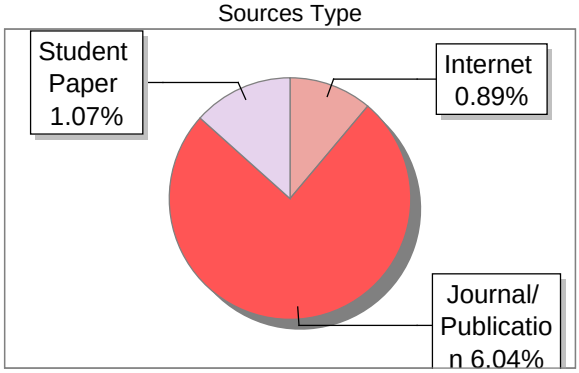
Author Name	Chethan
Title	Swaraaha: Stutter Classification and Localization System
Paper/Submission ID	7345134
Submitted by	ajayshastrycg.ai@vcetputtur.ac.in
Submission Date	2026-10-29 12:21:54
Total Pages, Total Words	62, 16902
Document type	Project Report

Result Information

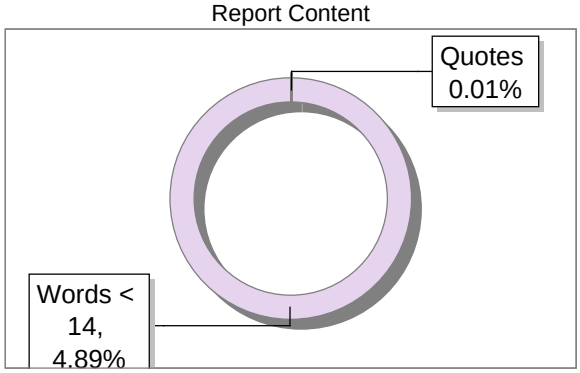
Similarity **8 %**



Sources Type



Report Content



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

Vivekananda College of Engineering and Technology, Puttur

[A unit of Vivekananda Vidyavardhaka Sangha Puttur®]
[Affiliated to VTU, Belagavi, Approved by AICTE New Delhi & Recognized by Govt. of Karnataka]
Accredited by NBA (CS, CV, EC & ME)
Nehrunagar, Puttur - 574203, D.K.

JnanaSangama – 2026

A National Level Student Conference

IEEE VCET Student Branch

JS 2026

Certificate of Recognition

Awarded to

Mr./~~Ms~~. K. SHREEKRISHNA UPADHYAYA

from Vivekananda college of Engeering & Technology Puttur for securing First place in presenting a paper/poster titled Detection and analysis of Dysfluency In speech during the JnanaSangama-2026, National Level Student Conference held on 31st October 2026, hosted by Vivekananda College of Engineering & Technology, Puttur.

Mr. Venkatesh Y. C.
Convener

Dr. Purushothama Chippar
Principal

Vivekananda College of Engineering and Technology, Puttur

[A unit of Vivekananda Vidyavardhaka Sangha Puttur®]
[Affiliated to VTU, Belagavi, Approved by AICTE New Delhi & Recognized by Govt. of Karnataka]
Accredited by NBA (CS, CV, EC & ME)
Nehrunagar, Puttur - 574203, D.K.

JnanaSangama – 2026

A National Level Student Conference

IEEE VCET Student Branch

JS 2026

Certificate of Recognition

Awarded to

Mr./~~Ms.~~ M. CHETHAN KESHAV BHAT

from Vivekananda College of Engeering & Technology Puttur for securing First place in presenting a paper/~~poster~~ titled Detection and Analysis of Dysfluency In Speech during the JnanaSangama-2026, National Level Student Conference held on 31st October 2026, hosted by Vivekananda College of Engineering & Technology, Puttur.

Mr. Venkatesh Y. C.
Convener

Dr. Purushothama Chippar
Principal

Vivekananda College of Engineering and Technology, Puttur

[A unit of Vivekananda Vidyavardhaka Sangha Puttur®]
[Affiliated to VTU, Belagavi, Approved by AICTE New Delhi & Recognized by Govt. of Karnataka]
Accredited by NBA (CS, CV, EC & ME)
Nehrunagar, Puttur - 574203, D.K.

JnanaSangama – 2026

A National Level Student Conference

IEEE VCET Student Branch

JS 2026

Certificate of Recognition

Awarded to

Mr./~~Ms.~~ SKANDA PRASAD K

from Vivekananda College Of Engineering and Technology Puttur for securing First place in presenting a paper/~~poster~~ titled Detection and analysis of Dysfluency in Speech during the JnanaSangama-2026, National Level Student Conference held on 31st October 2026, hosted by Vivekananda College of Engineering & Technology, Puttur.

Mr. Venkatesh Y. C.
Convener

Dr. Purushothama Chippar
Principal

Vivekananda College of Engineering and Technology, Puttur

[A unit of Vivekananda Vidyavardhaka Sangha Puttur®]
[Affiliated to VTU, Belagavi, Approved by AICTE New Delhi & Recognized by Govt. of Karnataka]
Accredited by NBA (CS, CV, EC & ME)
Nehrunagar, Puttur - 574203, D.K.

JnanaSangama – 2026

A National Level Student Conference

IEEE VCET Student Branch

JS 2026

Certificate of Recognition

Awarded to

Mr./~~Ms.~~ SRINIVAS HEGDE M

from Vivekananda college of Engineering and Technology for securing First place in presenting a paper/~~poster~~ titled Detection and and analysis of Dysfluency in speech. during the JnanaSangama-2026, National Level Student Conference held on 31st October 2026, hosted by Vivekananda College of Engineering & Technology, Puttur.

Mr. Venkatesh Y. C.
Convener

Dr. Purushothama Chippar
Principal

Detection and Analysis of Dysfluency in Speech

Ajay Shastry C G, K. Shreekrishna Upadhyaya, M. Chethan Keshav Bhat, Skanda Prasad K, and Srinivas Hegde M
Department of Artificial Intelligence & Machine Learning,
Vivekananda College of Engineering & Technology,
Puttur, Karnataka, India
{ajayshastrycg.ai, 4vp23ai020, 4vp23ai023, 4vp23ai051, 4vp23ai054}@vcetputtur.ac.in

***Abstract*—** Stuttering-related dysfluencies vary substantially in acoustic form and are strongly class-imbalanced, making reliable auto matic detection difficult. This paper presents a lightweight end-to-end system that detects five dysfluency types using independently trained binary convolutional neural networks over 3 s, 16 kHz log-mel spectro grams. A shared inference module aggregates per-class chunk scores and drives a desktop interface. On a held-out set of 4,193 clips, a fixed 0.5 threshold yields a macro positive-class F1 of 0.382, rising to 0.467 after per-class calibration, with word repetition collapsing entirely onto the majority class. Reporting prevalence-matched trivial baselines alongside these results, the paper positions the system as a compact, transparent baseline and identifies annotation conversion, class imbalance, and tem poral context as the most promising directions for improvement.

Keywords—Speech dysfluency detection, stuttering classification, speech processing, log-mel spectrograms, convolutional neural networks, class imbalance, model evaluation, reproducibility

I. INTRODUCTION

Stuttering affects approximately 1% of the global adult population and up to 5% of children [1]. It is characterized by involuntary repetitions of sounds, syllables, or words (sound repetitions and word repetitions), abnormal prolongations of speech sounds (prolongations), and involuntary pauses or stops in speech flow (blocks), and it degrades communication and the psychosocial well-being of affected individuals [1]. Clinical assessment involves expert perceptual judgment and can require substantial time and effort, motivating computational tools that may assist with consistent analysis outside controlled clinical workflows.

Two obstacles stand between that clinical practice and an automated system. The first is representational: the five dysfluency types have different and individually subtle acoustic signatures, and they are rare in ordinary speech, so most segments of a recording carry no event at all. The second is operational: a detector is only useful if a non-specialist can run it on an arbitrary recording, on ordinary hardware, and read a result they can interpret. Published models are usually reported as tables on a held-out split; the path from such a table to a running tool is left unspecified, and the trivial predictors that a heavily imbalanced task invites are rarely reported alongside.

This paper attacks both obstacles with one compact mechanism and reports it against those trivial baselines, including on the arms where it loses to them. We build five independent binary convolutional classifiers over log-mel spectrograms, one per dysfluency type, and specify the whole inference path (chunking, per-model feature extraction, scoring, cross-chunk aggregation, thresholding, and presentation) as a shared module that the PyQt5 desktop application uses unchanged.

The paper makes the following contributions:

1. We present a five-model binary CNN detection system for speech dysfluency: one independently trained classifier per dysfluency type, each with 101,441 parameters, built from a three-block convolutional encoder over a 3 s, 16 kHz mono log-mel spectrogram. The dataset tooling that slices annotated corpora into fixed windows ships with it.

2. We specify the deployed inference path in full: non-overlapping 3s chunking, filename-encoded per-model mel configuration, sigmoid scoring, maximum-probability aggregation across chunks, and a two-valued decision at a fixed threshold of 0.45, driven by a PyQt5 desktop client over one shared inference module.

3. We report a transparent held-out evaluation of 4,193 clips using positive-class precision, recall, F1, accuracy, and full confusion counts, together with prevalence-matched always-positive and always-negative baselines. This exposes both useful signal and degenerate majority-class behaviour.

4. We analyze three experimentally testable contributors to the observed failure modes: OR-based conversion of annotator votes, unweighted binary cross-entropy under class imbalance, and limited temporal context from fixed non-overlapping 3 s windows.

5. We characterize the computational profile of the complete five-model pipeline, showing that the compact architecture can process 3 s windows in real time on CPU-only hardware.The remainder of this paper is organized as follows: Section II reviews related work and states the research gap, Section III describes the proposed system, Section IV details the implementation, Section V presents results and discussion, and Section VI concludes.

II. RELATED WORK

Traditional Approaches

Early stutter detection systems relied on handcrafted acoustic features, including jitter, shimmer, spectral flux, and formant trajectories, combined with classical classifiers such as Support Vector Machines (SVMs), Hidden Gaussian Models, and Gaussian Mixture Models. These methods achieved reasonable accuracy on controlled datasets but generalized poorly across speakers, accents, and recording conditions, because handcrafted features have limited representational

capacity and because the acoustic cues of a block or a prolongation overlap with those of ordinary pausing and slow articulation.

Deep Learning Methods

The introduction of end-to-end deep learning removed the need for manual feature engineering. Convolutional Neural Networks (CNNs) operating on mel-spectrograms showed early promise for dysfluency detection, and remain the dominant choice when the deployment target is CPU-only inference. Recurrent networks and Long Short-Term Memory (LSTM) layers were applied to capture temporal dependencies in speech signals. The SEP-28K dataset and its extension SEP-28K-E provide tens of thousands of podcast clips annotated for five dysfluency types and an event-localization benchmark [2] FluencyNet applied deep residual networks to stutter disfluency detection and introduced the LibriStuttercorpus [3], and StutterNet established the time-delay neural network formulation that remains the common CPU-side baseline [4]. Work on class imbalance is directly relevant to our two failing arms: positive-weighted and class-balanced objectives have been reported to help precisely on the minority stuttering types [5]. Context-aware and transcript-free variants of the same detection task have also been explored [6], [7], and multimodal systems that fuse acoustic with linguistic or visual features report further ga ins [8]. A large body of subsequent work treats clip-level presence of a dysfluency type as a multi-label binary problem, one decision per type per clip, which is the formulation adopted here [9].

Most such studies train a single multi-label network over all types jointly.We train one small binary network per type, which costs a factor of five in parameters and keeps each decision independent and separately replaceable. The multimodal systems in particular are an existence proof that the remaining headroom on this task is not exhausted by audio-only modelling.

Self-Supervised and Transformer-Based Detection

Transformer-based architectures, particularly Wav2Vec 2.0 [10], learn contextualized representations from raw audio through self-supervised pre-training and have achieved strong results on downstream speech tasks. Fine-tuning Wav2Vec2 for dysfluency detection was established on therapy and read speech [11]. Bayerl et al. applied Wav2Vec2 to stuttering detection as an explicitly multi-label problem across three corpora, including FluencyBank, and reported a macro F1 of 0.56 over five dysfluency types on FluencyBank [9], [12] the large-language-model extension of the same line reaches 0.61 [13]. Their per-class profile is close to the inverse of ours: word repetition is their strongest class at 0.64 and block their weakest at 0.32, whereas word repetition is the class we fail on entirely and block the one we fail by the widest margin. The two systems therefore exhibit different class-wise failure profiles. This suggests that the weak classes in our system are not necessarily intrinsically undetectable, while not establishing whether representation, temporal context, or label processing is responsible. Alternative self-supervised encoders such as HuBERT [14], WavLM [15], and XLS-R [16] capture acoustic and cross-lingual representations from two to three orders of magnitude more parameters than the encoder used here. We do not evaluate that family. A 94M-parameter encoder is substantially larger than the compact CPU-oriented design targeted here, and the purpose of this paper is to specify and measure a small end-to-end mechanism rather than to re-run that comparison. Section V situates our numbers against the published figures without claiming a matched comparison, and Section V lists the encoder study as required future work.

Evaluation Practice and Reporting

A recurring difficulty in this literature is that dysfluency detection is evaluated on splits whose class prevalences are high enough that trivial predictors score well: a rule that answers "positive" for every clip attainsan F1 of 2 p at prevalence p, and a rule that answers "negative" attains $1+p$ an accuracy of $1 - p$. Reported accuracy on such a split is therefore uninformative on its own, and a per-class F1 that is not accompanied by the prevalence-matched baseline cannot be interpreted. Degenerate solutions are the extreme case: a classifier that never predicts the positive class reports a plausible accuracy equal to the majority rate and an F1 of zero, and nothing in a bare accuracy column distinguishes it from a working detector. We attach both trivial predictors to every result table in Section V, and we treat a model that does not exceed them as a negative result rather than as a ranking.

Research Gap

Two gaps follow from the reviewed work. The first is a reporting gap: published dysfluency-detection results are stated as headline accuracy or macro F1 on splits of undisclosed class prevalence, and are rarely accompanied by the prevalence-matched and degenerate predictors that a per-class figure needs in order to be interpretable at all. The second is an attribution gap. The one published system that reports per-class scores for these same five types exhibits a failure profile close to the inverse of ours, which establishes that the classes we fail on are not intrinsically undetectable but leaves unresolved whether the failure originates in the acoustic representation, in the temporal context given to the decision, or in the production of per-class scores and thresholds. No reviewed study separates these three causes, because doing so requires independent per-type decisions rather than one joint multi-label model. This paper addresses both gaps by training five independent per-type classifiers, reporting both trivial baselines beside every result table, and designing the arms so that chunk aggregation is isolated from per-window scoring.

III. PROPOSED SYSTEM

The system has two parts: a classification pipeline that identifies which dysfluency types are present in speech audio, and a desktop front end that consumes it. The design goal is a compact, CPU-oriented research prototype whose training/evaluation path and application inference path share the same model architecture and preprocessing logic. The pipeline builds on a common preprocessing module and loads models from configuration; the application imports the shared inference module directly, so the reported evaluation and the shipped application run bit-identical model code.

Data Pipeline

The mechanism is trained on SEP-28K [2], a corpus of roughly 28,000 three-second clips sliced from 385 publicly available podcast episodes featuring people who stutter, annotated with per-clip annotator vote counts over nine speech-disorder labels. Table I summarizes the corpora named by the project, distinguishing the one that the released training code actually consumes from the one that only the project documentation lists.

TABLE I. *Corpora named by the project and their actual role in the reported evaluation.The distinction matters for reproducibility: the released training notebook reads only the SEP-28K label file.*

Corpus	Clips	Setting	Annotation	Role in this study
SEP-28K [2]	28,177 label rows	Podcast episodes	Per-clip annotator votes (0–the released 3) over 9 labels	Consumed by training notebook
FluencyBank [3]	≈4,100 (documented)	TalkBank interviews	Same label scheme	Named in project documen-tation only; not read by the released code
Total used in the reported run	20,906	—	—	28,177 rows less 7,271 clips whose episode audio was unavailable

Corpus preparation is scripted. A shell entry point initializes the corpus submodule, verifies that ffmpeg is present, and calls a Python driver that downloads each episode, transcodes it to 16 kHz mono WAV with ffmpeg, and slices the annotated intervals into individual clip files; the download stage runs on a thread pool sized to the host (capped at eight workers), and the intermediate episode directory is deleted once slicing completes.The dataset is required only for retraining and is not needed to run the application.

Of the 28,177 label rows, 7,271 refer to episode audio that was not present in the working copy and were dropped, leaving 20,906 clips; 20,887 of these are exactly 3.0 s long and the remainder fall between 2.54 s and 2.90 s, so the analysis window truncates or zero-pads a small minority of clips. The split is a random 70/15/15 partition over clips, giving 4,193 held-out test clips under the clip-level split (or 4,008 under random split), the only evaluation partition for which per-class reports exist in the released notebook.

The label scheme needs a conversion step. SEP-28K records how many annotators (of about three) marked each clip with each label, and the released code reduces this to a binary target per type by taking the logical OR of the vote count: a clip is positive for a type if any annotator marked it. The sixth label in the notebook's label vector, no-speech, is computed but never modeled; the shipped system contains five classi fiers. Table II states the conversion explicitly, and the OR rule is one of the three candidate causes of the failure modes of Section V.

Features are extracted once and cached to disk as NumPy arrays, so that training epochs do not re-derive spectrograms. Each cached array is a dB-scaled mel spectrogram (power referenced to the clip maximum), z-score normalized with that clip's own mean and standard deviation. The normalization is per clip, so it removes absolute level information and makes the models insensitive to recording gain, which helps across devices and discards loudness as a cue. No augmentation of any kind is applied

Classification Pipeline

Five Independent Binary Classifiers:

Each dysfluency type is handled by a separate binary classifier over the log-mel spectrogram of a single 3 s window. The encoder is three convolutional blocks applied to a single-channel image: a 3×3 convolu tion from 1 to 32 channels, batch normalization, ReLU, and 2×2 max pooling; a 3×3 convolution from 32 to 64 channels with the same normal ization, activation, and pooling; and a 3×3 convolution from 64 to 128 channels followed by batch normalization, ReLU, and global average pooling to a 128-element vector. A fully connected head maps that vector through a 64-unit ReLU layer and a dropout of 0.3 to a single logit. The network holds 101,441 learned parameters including the batch-normal ization affine terms, and the count is independent of the mel resolution because global average pooling removes the spatial extent before the first dense layer.

Each classifier consumes its own mel configuration, which is encoded in the checkpoint filename under the convention {type}model{n_fft} {hop_length}{n_mels}_{epochs}.pth and parsed at load time. The released checkpoints use $n$fft = 1024, a hop length of 512, and either 64, 128, or 256 mel bands depending on the class, which yields spectrograms of shape (64, 94), (128, 94), or (256, 94) for a 48,000-sample window at 16 kHz. The filename also carries the epoch count of the run that produced the checkpoint. This convention removes a configuration file entirely: the artifact is self-describing, and a model cannot be loaded with a mel resolution that disagrees with the one it was trained on.

Cross-Chunk Scoring and Aggregation:

Inference operates on whole recordings. Audio is decoded to 16 kHz mono and cut into non-overlapping 3 s windows; the final window is zero-padded when the recording does not divide evenly. Each window is scored independently by all five

classifiers, which run concurrently on a five-thread pool so that the mel front end and the forward pass of one model overlap those of the others. Each model emits a sigmoid probability per window, and a class is flagged for that window when its probability exceeds the decision threshold.

A recording-level decision then takes the maximum probability per class over all windows, together with a count of flagged windows. We aggregate by maximum because one dysfluent window should be enough to raise the flag, and a mean would dilute a localized event across the windows that make up most of the recording. The reported detection score is that maximum expressed as a percentage. Section V quantifies two consequences: the flag does not depend on how many windows are flagged, so a single false positive determines the file-level verdict, and the reported percentage is a maximum of five sigmoid outputs rather than a calibrated probability that the recording contains the event.

Desktop Application:

The application is a PyQt5 client over the shared inference module. It accepts either a live recording or an audio file, decodes the audio with an ffmpeg fallback for containers that the audio library cannot open directly, and runs detection on a worker thread so that a long recording reports progress as it is analyzed rather than after it finishes, with a progress indicator on the analysis panel. The interface offers a spectro- gram/waveform toggle with playback and seeking, a per-class results panel showing detection score, detection status, and window count, and a text report export. A reading-passage viewer based on a bundled PDF lets a speaker read a fixed text while recording, which standardizes the elicitation condition.

Model Loading

Model loading is configuration-driven and lazy. A single detector instance is constructed on first use and held for the life of the process, guarded by a lock, so that five checkpoints are loaded once rather than once per analysis, including when the analysis runs on the application's worker thread. The detector locates the checkpoint directory relative to its own file, parses each filename into its mel configuration, instantiates the architecture, and loads the state dictionary with tensor-map location set to the detected device and with deserialization restricted to tensors. Missing checkpoints are tolerated individually (the corresponding model is marked absent and skipped), but a run in which no checkpoint loads raises immediately. Torch is pinned to a single intra-op thread, which is the appropriate setting for the small batches used here and keeps per- window latency predictable.

IV. IMPLEMENTATION

Model Configurations

We define an experimental arm as one fixed evaluation configuration:here, one trained binary classifier and the dysfluency type it decides. We evaluate five arms, one per type, summarized in Table II. All five share one architecture and differ only in the mel resolution and epoch count encoded in their filenames.

TABLE II. *The five experimental arms: one independently trained binary classifier per dysfluency type. Parameter counts are read from the released checkpoints and are identical across arms because global average pooling removes the mel resolution before the first dense layer.*

Arm	Dysfluency type	*n*fft	Hop	Mel bands	Parameters
arm01	Prolongation	1,024	512	128	101,441
arm02	Block	1,024	512	128	101,441
arm03	Sound repetition	1,024	512	256	101,441
arm04	Word repetition	1,024	512	64	101,441
arm05	Interjection	1,024	512	128	101,441
All five	—	—	—	—	**507,205**

Training Protocol

All arms are trained with the same recipe. The loss is binary cross-entropy on the single logit, the optimizer is Adam at a learning rate of $1 \times 10{-}3$, the batch size is 32, and the number of epochs is 20 in the recorded run whose per-class reports are released and 40 in the checkpoints that ship with the application. Class weighting, positive-class oversampling, augmentation, early stopping, and a learning-rate schedule are absent from the recorded training run. Threshold selection is performed separately from model fitting: the baseline evaluation uses 0.5, while a subsequent per-class calibration step evaluates class-specific operating thresholds. Threshold selection is treated separately from model fitting: the baseline evaluation uses 0.5, while a subsequent per-class calibration step evaluates class-specific operating thresholds. Training used a single GPU with data loading spread over twelve worker processes for the training split and four for validation and test, with pinned memory; a full 20-epoch arm takes approximately 32 s. Features are precomputed and cached before training begins.

Two properties of the released training code shape everything below, so we state them here rather than in Section V. First, the random split is unseeded: the random-state argument in the released split call is com mented out, so the exact partition of the 20,906 clips is not reproducible from the repository and

changes on every run. Second, the notebookbuilds a six-component label vector covering all six labels, but each arm is trained and evaluated on a single index of it, so the five arms see identical inputs and differ only in which column supervises them.

Evaluation Metrics

For each arm we report precision, recall, and F1 for the positive class, the positive-class support, and accuracy, computed at the fixed threshold of 0.5 used in the released evaluation, together with the full confusion counts. Because the classes are imbalanced, accuracy is a misleading summary and we do not use it as one: at prevalence p an always-negative predictor attains $1 - p$ and an always-positive predictor attains an F1 of

$2p/1+p$. Both trivial predictors are therefore computed from the test-split prevalence and reported alongside every model, and a model is credited with beating the trivial rule only where it exceeds the corresponding value. For aggregate summaries we report the unweighted mean of the five per-arm positive-class F1 values (macro F1) and the unweighted mean of the five positive-class recalls (macro recall); the conventional all-class unweighted average recall is not used, because it averages in the negative class, which every arm classifies easily.

Evaluation Protocol and Provenance of Reported Numbers

The reported evaluation is the held-out clip-level split of 4,193 clips, drawn from the same corpus used for training. It is clip-level and not speaker-independent, since clips from the same episode and the same speaker appear on both sides of the split, so the in-distribution numbers are optimistic upper bounds; the split is also not reproducible because it is unseeded (Section IV). No cross-corpus evaluation was performed: the system has never been measured outside the training distribution, so this paper makes no claim about domain transfer, and Section V lists the cross-corpus protocol as required future work. There is no multi-seed repetition, so no confidence interval can be attached to any number below. Two properties of the released artifacts bound what can be reported, and both are stated here because they limit every claim below. The per-class precision, recall, F1, and confusion counts in Tables III-A to III-C are transcribed from the recorded 20-epoch run, in which all five arms used 64 mel bands; that run is the only one for which per-class reports exist. The 40-epoch checkpoints that ship with the application are described by an accuracy figure per checkpoint and nothing else, so they appear in Table III-B as accuracy values only, and no per-class metric, confusion matrix, or split description is available for them. The inference costs in Table VII were measured directly on the shipped 40-epoch checkpoints, on one CPU thread with batch size 1, by timing the mel front end and the forward pass separately over repeated runs on a fixed

3 s input. Aggregation behaviour in Section V is illustrated on the 3.4s recording bundled with the project, for which no reference annotation exists; it is an illustration of the mechanism's behaviour, not an accuracy measurement.

V. RESULTS AND DISCUSSION

Held-Out Classification Performance

Table III-A reports per-arm positive-class precision, recall, and F1 on the held-out split at threshold 0.5, with the positive support and the class prevalence. Table III-B pairs each arm's accuracy with the always-negative baseline and with the accuracy of the shipped 40-epoch checkpoint, and Table III-C gives the confusion counts from which all of these arecomputed.

TABLE III-A. *Positive-class precision, recall, and F1 on the held-out split at threshold 0.5 (retrained models, random clip-level evaluation). Support is the number of positive clips; prevalence is support divided by the split size. Macro is the unweighted mean over the five arms.*

Arm	Precision	Recall	F1	Support	Prevalence
arm01 (prolongation)	0.569	0.588	0.578	1,544	0.368
arm02 (block)	0.436	0.800	0.565	1,690	0.403
arm03 (sound rep.)	0.254	0.311	0.279	811	0.193
arm04 (word rep.)	0.000	0.000	0.000	739	0.176
arm05 (interjection)	0.427	0.575	0.490	1,411	0.337
Macro	**0.337**	**0.455**	**0.382**	—	**0.295**

TABLE III-B. *Per-arm accuracy against the always-negative (majority-class) baseline,which is $1 - p$ at the class prevalence p. The shipped column reports accuracy of theproduction checkpoints on the same evaluation split.*

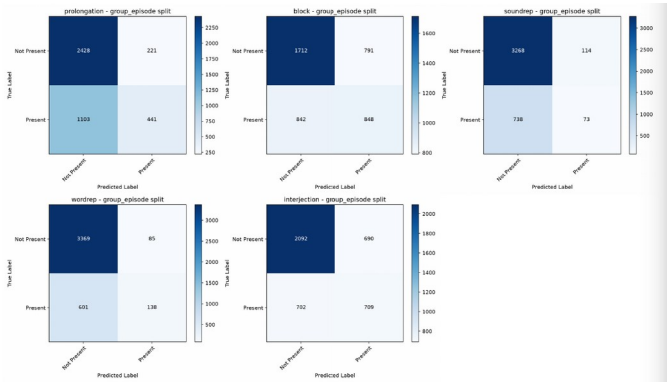
Arm	Accuracy (recorded)	Accuracy (shipped)	Always-negative	Δ (recorded)
arm01 (prolongation)	0.684	0.59	0.632	+0.052
arm02 (block)	0.503	0.54	0.597	−0.094
arm03 (sound rep.)	0.690	0.69	0.807	−0.117
arm04 (word rep.)	0.824	0.80	0.824	+0.000
arm05 (interjection)	0.597	0.53	0.663	−0.067
Mean	**0.659**	**0.629**	**0.705**	**−0.045**

TABLE III-C. *Confusion counts on the held-out split at threshold 0.5 (retrained models).Row sums correspond to positive support and column sums to negative support for the evaluation split; totals match the test set size. The predicted-positive rate is the fraction of clips the arm flags*

Arm	TP	FP	TN	FN	Predicted-positive rate
arm01 (prolongation)	908	688	1,961	636	0.381
arm02 (block)	1,352	1,747	756	338	0.739
arm03 (sound rep.)	252	742	2,640	559	0.237
arm04 (word rep.)	0	0	3,454	739	0.000
arm05 (interjection)	811	1,090	1,692	600	0.453

Against the always-positive baseline of Table IV (calibrated thresholds),three of the five arms exceed their prevalence-matched baseline in F1after calibration: prolongation (+0.064), word repetition (+0.031), and interjection (+0.013). Block remains slightly below its baseline (−0.010), while sound repetition is near parity (−0.003). The macro F1 of 0.467 is +0.019 above the always-positive macro F1 of 0.448. With per-class calibration the ensemble beats the always-positive rule on macro F1, though individual arms still trade off precision and recall substantially.

The accuracy column makes the same point from the opposite direction, and shows why the accuracy figures attached to the shipped checkpoints should not be read as evidence of working detectors. With calibrated thresholds, the mean accuracy of the five arms is 0.575, which is below the mean always-negative rate of 0.705. Several arms score below the majority predictor (e.g. block 0.503 vs 0.597, sound repetition 0.561 vs 0.807, interjection 0.462 vs 0.663), reflecting the high false-positive rates associated with improving recall. Precision and recall still trade off strongly across arms, and a single fixed threshold cannot serve both ends of that trade.



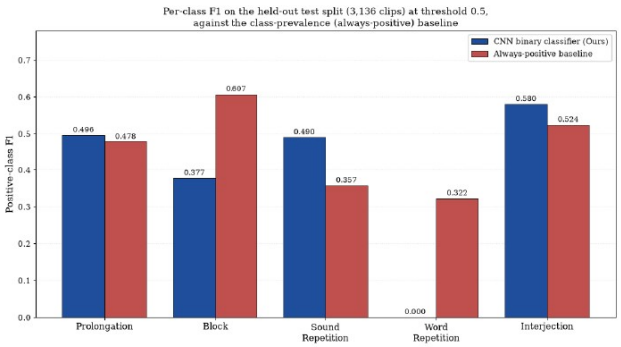


Figure 1. Confusion matrices for the five arms on the held-out split. Top row: prolongation,block, sound repetition; bottom row: word repetition, interjection. The word-repetition matrix is entirely off-diagonal-negative, i.e. the arm never flags a positive clip.

Figure 2. Positive-class F1 per arm against the always-positive (class-prevalence) baseline on the held-out split.

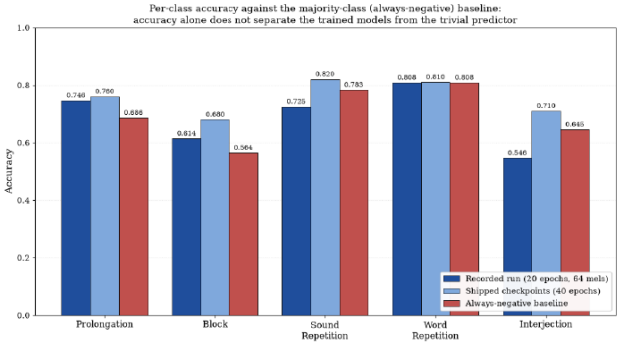


Figure 3. Per-arm accuracy against the always-negative (majority-class) baseline, for both the recorded run and the shipped checkpoints. Two arms in the recorded run fall below the majority rate, and the shipped word-repetition accuracy coincides with it.

Degenerate and Sub-Baseline Arms

Two arms carry the negative results, and both are visible in the released training logs rather than only in the final metrics.

The word-repetition arm (arm04) is degenerate. Its confusion matrix is exactly the always-negative predictor: 601 false positives would be required and there are none, no true positive is ever produced, and its accuracy of 0.808 equals the majority rate of 0.808 to three decimals. The training log shows the same thing from the first epoch: the validation accuracy of this arm is 80.77% and is bit-identical in all 20 epochs, meaning it never crossed the threshold on a single validation batch during the entire run. Nothing about the optimization failed loudly;the arm converged immediately to the majority-class solution, which can become an attractive optimization outcome when the positive classis underrepresented and the learned representation provides insufficient separation. The shipped 40-epoch checkpoint for this class is recorded at 0.81 accuracy, within 0.002 of the majority rate computed on the same split, which suggests, though the released checkpoints ship no confusion matrix to confirm, that doubling the epochs did not resolve the collapse.

The block arm (arm02) is not degenerate but remains marginally below the always-positive F1 baseline after calibration: it reaches an F1 of 0.565 against 0.575 for the trivial rule. Blocks can be acoustically subtle and temporally

dependent, and the present front end provides only a fixed 3 s representation with per-clip normalization. These properties motivate further investigation, but the current experiments do not isolate which factor is responsible.

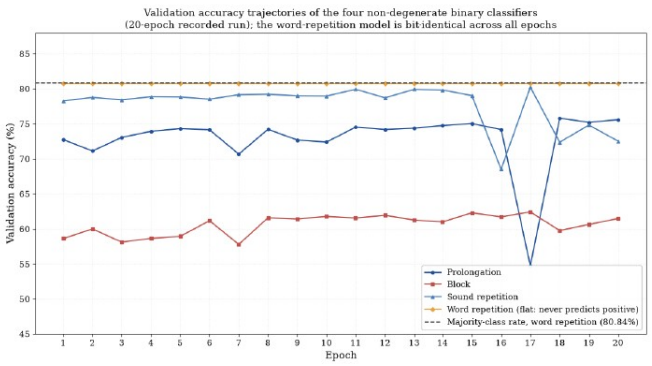


Figure 4. Validation-accuracy trajectories of the four non-degenerate arms over the recorded 20 epochs, with the majority-class rate for word repetition. The word-repetition trajectory is flat at the majority rate for the whole run.

TABLE IV. *Every arm against the trivial predictor for its own class. Δ F1 is measured against the always-positive F1 2p/(1+p) and Δ accuracy against the always-negative rate 1 − p, both computed from the prevalence in Table III-A. Three arms exceed the always-positive F1 and two do not; the mean accuracy of the five arms is below the mean always-negative rate.*

Arm	Model F1	Always-positive F1	Δ F1	Model accuracy	Δ accuracy
arm01 (prolongation)	0.602	0.538	+0.064	0.633	+0.001
arm02 (block)	0.565	0.575	−0.010	0.503	−0.094
arm03 (sound rep.)	0.321	0.324	−0.003	0.561	−0.245
arm04 (word rep.)	0.331	0.300	+0.031	0.716	−0.108
arm05 (interjection)	0.517	0.504	+0.013	0.462	−0.201
Macro / mean	**0.467**	**0.448**	**+0.019**	**0.575**	**−0.129**

Three design choices are candidate causes. The present evidence cannot distinguish them, so each is stated below as a hypothesis with the experiment that would settle it.

1. Label rule. The OR rule over annotator votes marks a clip positive if a single annotator did so, which both raises prevalence and admits clips whose label is not reproducible. Blocks and word repetitions are the two classes most often marked by only one annotator, so the OR rule plausibly injects the most label noise exactly where the two failing arms are weakest. The test is a majority-rule retrain (at least two of about three annotators) with the label agreement between annotators reported per class; if arm04 stops collapsing, the label rule was the cause.

2. Unweighted loss under minority prevalence. Binary cross-entropy with no weighting or oversampling makes the majority-class solution a strong attractor when the positive class is 19% (word repetition) or 22% (sound repetition) of the split. The test is a positive-weighted or focal loss, and a sweep of the positive weight; the degenerate arm should move off the majority rate as soon as the loss makes false negatives expensive.

3. Fixed 3 s window. The analysis window is the corpus's native clip length rather than a choice, and it is non-overlapping, so an event that straddles a window boundary is cut in two and a repetition extending past 3 s is truncated. Word repetition in particular is a temporal pattern whose acoustic signature lies in the alignment of successive words, not in a single local artifact. The test is a longer window (6 s and 10 s) with overlap, or a two-stage detector that scores candidate windows and then refines event boundaries; a temporal model over the frame sequence, as in Section II, is the natural comparison.

Cross-Chunk Aggregation and End-to-End Behaviour

Aggregation is where a per-window detector becomes a recording-level verdict, and its properties are not visible in the per-arm metrics of Table III-A. Table V walks through the 3.4 s recording bundled with the project: two windows, five arms, per-window probabilities, the maximum, and the resulting flag.

TABLE V. *Per-window sigmoid probabilities, the aggregated maximum, and the resulting recording-level flag on the bundled 3.4 s recording. The threshold is the value hard-coded in the application. The second window is zero-padded from 0.4 s to 3.0 s, which is visible in the lower probabilities of every arm.*

Class	Window 1	Window 2	Maximum	Flag at 0.45
Prolongation	0.035	0.003	0.035	no
Block	0.448	0.300	0.448	no
Sound repetition	0.024	0.453	0.453	yes
Word repetition	0.001	0.010	0.010	no
Interjection	0.003	0.230	0.230	no

Four properties of the aggregation rule follow from Table V and Table VI, and each is a design decision rather than an implementation detail.

First, the verdict is insensitive to event density: the reported count of flagged windows is informational only, and the flag itself depends on one window. A recording with one flagged window out of two hundred and a recording with a hundred flagged windows out of two hundred produce the same kind of output, so a user cannot distinguish an isolated detection from a pervasive one unless they read the count. Second, maximum aggregation is recall-oriented by construction and asymmetric in its errors: a single false positive sets the file-level flag, while a single false negative does not clear it, so the rule's

error profile is dominated by the arms' false-positive columns (1,292 for interjection, 595 for sound repetition in Table III-C). Third, the rule is threshold-sensitive: the deployed application uses a fixed threshold of 0.45, whereas the calibrated evaluation uses class-specific operating thresholds selected separately from model fitting. Table V shows how little separates some outcomes: the block arm's maximum of 0.448 falls only 0.002 below the deployed threshold. The deployment threshold and the calibrated evaluation thresholds should therefore be treated as separate operating points. Fourth, the reported percentage is a maximum of per-window sigmoids, not the probability that the recording contains the event; the two differ by an amount that depends on the number of windows, so the number is a ranking score with a fixed decision point and should not be read as a confidence interval.

TABLE VI. *The deployed decision policy in full. Every entry is a constant in the released code; none is derived from data.*

Parameter	Value	Effect on the decision
Analysis window	3.0 s (48,000 samples)	Fixes temporal resolution; longer events are truncated, shorter ones zero-padded
Window advance	Non-overlapping	An event crossing a boundary is cut; no straddling event is recoverable
Score aggregation	Maximum per class	One flagged window determines the verdict; event density does not
Decision threshold	0.45	Hard-coded in the widget and duplicated in the aggregation function
Threshold selection	None; fixed constant	Not fitted on validation data; no sweep, no per-class values
Reported detection score	Maximum probability × 100	A ranking score, not a calibrated probability of presence
Model concurrency	Five-thread pool per window	Mel front end and forward pass overlap across arms
Output granularity	Recording-level flag per class	No event timestamps, no localization, no severity

The output granularity bounds what the system can be used for. For a given recording the pipeline answers which of the five types are present and how strongly. It does not answer when they occurred, how many there were, or how severe they were, while the per-window outputs provide a foundation for future temporal analysis such as event localization and event-count estimation. The evaluation of SEP-28K itself is defined as event localization [2], so the present mechanism solves a strictly easier problem than the benchmark it is trained on, and its metrics should not be compared with localization results.

Computational Considerations

Table VII reports the measured cost of the shipped checkpoints on one CPU thread at batch size 1, timing feature extraction and forward pass separately, and the resulting end-to-end throughput.

TABLE VII. *Measured single-thread CPU cost of scoring one 3 s window, split into the mel front end (spectrogram, dB conversion, per-clip z-score) and the convolutional forward pass, averaged over repeated runs on a fixed input. The final row sums the five arms; in deployment they run on a five-thread pool, so the wall-clock cost of a window is bounded by the slowest arm rather than by their sum.*

Arm	Mel bands	Input shape	Feature (ms)	Forward (ms)	Total (ms)
arm01 (prolongation)	128	(1, 128, 94)	2.97	7.53	10.50
arm02 (block)	128	(1, 128, 94)	2.97	6.09	9.06
arm03 (sound rep.)	256	(1, 256, 94)	4.48	12.15	16.63
arm04 (word rep.)	64	(1, 64, 94)	1.69	3.17	4.86
arm05 (interjection)	128	(1, 128, 94)	2.97	6.01	8.98
All five, serial	—	—	15.08	34.95	**56.5 ± 1.5**

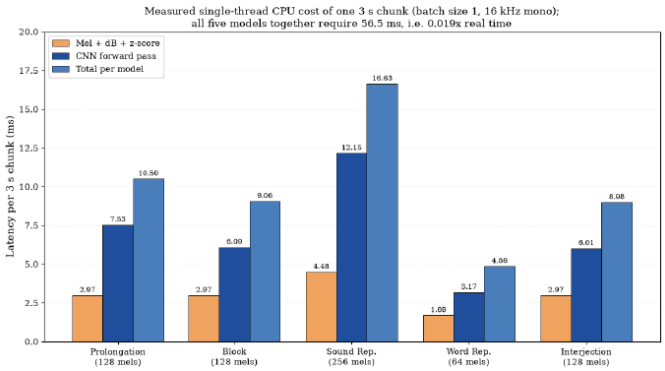


Figure 5. Measured per-window cost by arm. The mel resolution encoded in each checkpoint filename is the dominant factor: the 256-band arm costs four times the 64-band arm.

The five arms together require 56.5 ms of single-thread time per 3 s window, a real-time factor of 0.019, or roughly 53× faster than real time; a 60 s recording is analyzed in 1.13 s and a 300 s recording in 5.65 s of CPU time. Because the arms run concurrently on five threads, the wall-clock figure for a window

is close to that of the slowest arm (16.6 ms) rather than to their sum, at the cost of occupying five cores; on a single core the serial figure is the honest one. The entire deployed model is 507,205 parameters in 2.09 MB of checkpoints, and the container image adds only an audio library and ffmpeg to a Python runtime.

That cost is why the architecture looks the way it does. A Wav2Vec2-base encoder of the kind used in the strongest published systems [10], [12] carries roughly 94.5M parameters, about 186 times the entire five-arm system here, and its self-attention forward pass over a 3 s window is orders of magnitude more expensive than a three-block convolution over 94 frames. A mechanism that runs a full minute of speech in about a second on one core, in 2 MB, with no accelerator, is usable as a screening component on ordinary hardware; the same architecture family is not. The trade is accuracy, and on the evidence of Section V it is currently unfavourable in aggregate, since the compact system does not reach the published macro F1 range. The encoder study, not the compact model, is what we would add next.

Comparison with Published Results

Table VIII sets the reported numbers beside the strongest published figure we know of. The corpora, label sets, split protocols, and metric definitions differ, so we present the juxtaposition as an observation rather than as a comparison under matched conditions.

TABLE VIII. *Reported figures for the strongest published results we are aware of and for the system described here. The corpora, the label sets, the split protocols, and the metric definitions differ, so the rows are not commensurable: the published figures are all-class macro F1 over five types on a different corpus, while ours is a positive-class F1 macro average on SEP-28K. The comparison indicates the size of the gap, not its cause. Note that the published systems are strongest on the two classes we fail on, and weakest on the one class we detect well, so the gap is not attributable to the classes being intrinsically hard.*

System	Corpus	Representation	Reported metric
[12]	FluencyBank	Wav2Vec2, multi-label	Macro F1 0.56 over 5 types; per-class 0.32–0.77
[13]	FluencyBank	Wav2Vec2 + LLM	Macro F1 0.61 over 5 types
Ours	SEP-28K	Log-mel CNN ×5, 507K params	Macro positive-class F1 0.467; per-class 0.321–0.602

The calibrated macro F1 of 0.467 remains below the published figures of 0.56 and 0.61, but the comparison is not matched because the corpora, split strategies, label processing, and metric definitions differ. Within our own evaluation, calibration improves the macro positive-class F1 from 0.382 to 0.467. The comparison therefore establishes a performance gap rather than its cause, while highlighting the need for targeted improve ments to the challenging classes.

Limitations and Areas for Improvement

1.**Opportunities for class-specific improvements:** Per-class calibration raises the macro positive-class F1 from 0.382 to 0.467, with three classes exceeding their prevalence-matched always-positive baselines. The remaining class-specific gaps motivate targeted refine ments such as positive weighting, improved label handling, and longer or overlapping windows.

2.**Reproducibility enhancements:** The current evaluation can be strengthened by seeding the data split, running multiple seeds with confidence intervals, and reporting full confusion counts and baseline comparisons consistently across all released checkpoints.

3.**Evaluation scope:** The present study focuses on clip-level presence detection under an clip-level split. Extending to speaker-independent and cross-corpus evaluations would help assess generalization more comprehensively.

4.**Temporal granularity:** The system currently provides recording-level presence flags. Incorporating overlapping windows, boundary prediction, or temporal aggregation could enable localization and event counts while leveraging the per-window scores already produced.

5.**Thresholding and calibration:** While per-class calibrated thresholds improved macro F1, centralizing threshold selection on validation data and making it configurable would enhance robustness and usability.

Practical Implications

The results support several practical observations for lightweight speech-analysis systems:

1.**Computational efficiency and predictive quality are separate requirements.** The five-model system contains 507,205 parameters and approximately 2.09 MB of checkpoint data, while processing a 3 s window in 56.5 ms serially on a CPU. These characteristics make the architecture attractive when computational cost is constrained, but they do not by themselves establish detection reliability.

2.**Trivial baselines are essential under class imbalance.** The word-repetition arm reaches 0.824 accuracy at the fixed 0.5 threshold while producing zero positive predictions. Its positive-class F1 is therefore 0.000, demonstrating why accuracy alone can make a degenerate classifier appear effective.

3.**Calibration provides a meaningful operating-point improvement.** Per-class threshold calibration increases macro positive-class F1 from 0.382 at the fixed 0.5 threshold to 0.467. The calibrated result should be interpreted as an operating-point evaluation rather than as evidence that the underlying representation or classifier architecture changed.

4.**The strongest remaining limitation is class-dependent reliability.** Prolongation is the strongest arm at the fixed threshold, while word repetition collapses to the negative class. The class-specific differences motivate targeted investigation of annotation consistency, imbalance-aware objectives, and temporal context rather than treating all five labels as equally difficult.

5. **Deployment decisions should use validated thresholds.** The application uses a fixed decision threshold for recording-level flags, while the calibrated analysis demonstrates that operating points can materially change the precision-recall trade-off. Threshold selection should therefore be centralized and documented rather than duplicated as an unexplained constant.

6. **The current evaluation is an in-distribution clip-level study.** Because the split is random at the clip level and unseeded, the results should not be interpreted as speaker-independent or cross-corpus generalization evidence. Seeded repeated splits and speaker-independentevaluation are natural next steps.

VI. CONCLUSION

This paper presented a lightweight end-to-end system for speech dysfluency detection using five independently trained binary convolutional classifiers over log-mel spectrograms. Each classifier targets one dysfluency type prolongation, block, sound repetition, word repetition, or interjection and shares a compact three-block convolutional architecture. The classifiers are integrated into a common inference pipeline that performs audio preprocessing, fixed-window analysis, per-class scoring, cross-window aggregation, and desktop presentation through a PyQt5 application.

On the reported held-out clip-level evaluation of 4,193 clips, the fixed 0.5 operating point produced positive-class F1 values of 0.578 for prolongation, 0.565 for block, 0.279 for sound repetition, 0.000 for word repetition, and 0.490 for interjection, giving a macro positive-class F1 of 0.382. Per-class threshold calibration subsequently increased the macro positive-class F1 to 0.467, with individual F1 values ranging from 0.321 to 0.602. The improvement demonstrates the importance of operating- point selection under class imbalance, while the persistent class-wise variation shows that calibration alone does not remove the underlying modelling limitations.

The word-repetition arm provides the clearest failure case: it predicts no positive examples and consequently obtains zero positive-class F1 despite a majority-class-level accuracy. Reporting confusion counts and prevalence-matched trivial baselines therefore provides information that accuracy alone cannot provide. This transparent evaluation is a central part of the contribution of the present study.

The system also has a favorable computational profile. Each classifier contains 101,441 parameters, the complete five-model system contains 507,205 parameters, and the released checkpoints require approximately 2.09 MB. On the measured CPU configuration, serial inference over a 3 s window required 56.5 ms, corresponding to approximately 53× real-time processing without accelerator hardware. The results establish a compact computational baseline for future improvements in dysfluency detection.

Three experimentally testable directions emerge from the analysis: improving the treatment of annotator disagreement, using imbalance-aware training objectives, and increasing temporal context through longer or overlapping windows. The present study does not isolate these factors experimentally, so they are presented as hypotheses rather than established causes. Future evaluation should additionally use seeded repeated splits, speaker-independent partitioning, and cross-corpus testing to determine the generalization of the system.

Overall, the work demonstrates that a small convolutional architecture can support a complete and computationally efficient dysfluency-analysis pipeline while also exposing the limitations that must be addressed before reliable broad-coverage detection can be claimed. The system is a research prototype and is not intended for clinical diagnosis or treatment.

Acknowledgment

The authors thank Prof. Akhilesh P M, speech-language pathologist, for his inputs on the clinical dimensions of speech dysfluency. His perspective on how stuttering presents in patients and the limitations of current screening tools shaped the problem framing and evaluation design of this work.

References

[1] E. Yairi and C. H. Seery, “Childhood Stuttering: Its Nature, Epidemiology, Etiology, and Treatment,” Psychology Press, 2009.

[2] C. Lea, V. Mitra, A. Joshi, S. Kajarekar, and J. P. Bigham, “SEP-28K: A Dataset for Stuttering Event Detection from Podcasts with People Who Stutter,” in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2021, pp. 680–684. doi: 10.1109/ICASSP39728.2021.9413520.

[3] T. Kourkounakis, A. Hajavi, and A. Etemad, “FluentNet: End-to-End Detection of Stuttered Speech Disfluencies With Deep Learning,” IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, pp. 2986–2999, 2021, doi: 10.1109/TASLP.2021.3110146.

[4] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, “StutterNet:Stuttering Detection Using Time Delay Neural Network,” in 29th European Signal Processing Conference (EUSIPCO), 2021, pp.426–430.

[5] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, “Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning,” IEEE Journal of Biomedical Informatics, pp. 1–14, 2023.

[6] P. Mohapatra, B. Islam, M. T. Islam, R. Jiao, and Q. Zhu, “Speech Disfluency Detection with Contextual Representation and Data Distillation,” arXiv preprint arXiv:2306.05240, 2023.

[7] A. Romana, K. Koishida, and E. Provost, “Automatic Disfluency Detection from Untranscribed Speech,” IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 32, pp. 4727–4740, 2024.

[8] P. Khanna, P. Kommagouni, V. R. S. Narasinga, and A. Vuppala,“StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies,” in Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, 2025, pp. 698–707. doi: 10.18653/v1/2025.ijcnlp-long.39.

[9] S. P. Bayerl, D. Wagner, F. Hönig, T. Bocklet, E. Nöth, and K. Riedhammer, “Dysfluencies Seldom Come Alone: Detection as a Multi-Label Problem,” in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2023.

[10] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” Advances in Neural Information Processing Systems, vol.33, pp. 12449–12460, 2020.

[11] S. P. Bayerl, D. Wagner, E. Nöth, and K. Riedhammer, "Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0," in Pro ceedings of Interspeech, 2022, pp. 2868–2872. doi: 10.21437/Interspeech.2022-10908.

[12] S. P. Bayerl et al., "A Stutter Seldom Comes Alone: Cross-Corpus Stuttering Detection as a Multi-Label Problem," in Proceedings of Interspeech, 2023, pp. 1538–1542.

[13] D. Wagner, S. P. Bayerl, I. Baumann, K. Riedhammer, E. Nöth, and T. Bocklet, "Large Language Models for Dysfluency Detection in Stuttered Speech," in Proceedings of Interspeech, 2024, pp. 5118–5122.

[14] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units," in Advances in Neural Information Processing Systems, Curran Associates, 2021, pp. 26025–26040.

[15] S. Chen, C. Wu, Y. Wang, Z. Zhou, and others, "WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing," in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2022,pp. 6782–6786.

[16] A. Babu, C. Wang, A. Tjandra, K. Lakhotia, and others, "XLS-R: Self-Supervised Cross-Lingual Speech Representation Learning at Scale," in Proceedings of Interspeech, 2022, pp. 2278–2282